

大语言模型与生成式基础模型

基础、系统、对齐与应用

尹诚

2026 年 5 月 20 日

前言

本书是《Large Language and Generative Foundation Models: Foundations, Systems, Alignment, and Applications》的中文可读版。中文稿以原英文稿为蓝本，在保持技术含义、章节顺序、术语边界和引用依据一致的前提下，使用更适合中文读者的表达方式组织句子。它不是“核心论点摘要”，也不是只保留结论的缩写版；它的目标是让中文读者能够沿着同一条论证路径，理解每一章为什么这样安排、每个概念解决什么问题、每种工程选择带来什么代价。

本书坚持一个基本立场：大语言模型不是一个孤立算法，而是一套由数据、目标函数、模型结构、系统实现、推理策略、评测协议和治理约束共同定义的工程系统。随着多模态大模型、统一理解-生成模型、扩散语言模型、视频生成模型、Omni 任意到任意接口和 vision-language-action 模型的发展，这个立场还需要进一步扩展：文本自回归只是生成式基础模型的一种重要形式，而不是整个领域的边界。理解现代基础模型的关键不是记住模型家族名称，而是能说清楚一个选择优化了什么、牺牲了什么、如何测量，以及测量本身可能在哪里失真。

本书文字、结构、例子和论证为原创教材性写作。书中研究判断尽量连接到公开论文、技术报告或课程材料。第三方幻灯片、视频转写、数据样本、代码清单和图像不作为正文内容复制。

伦理与来源说明

本书只复述公开研究中的概念、方法和结论，不复制第三方课程材料、私有数据、模型输出、代码或图像。涉及模型报告、数据集、软件项目、法规和政策文件时，均把它们作为参考来源，而不是作为可直接搬运的内容。

读者在复现实验时应特别关注数据来源、许可证、个人信息、评测污染和安全边界。一个可以训练的系统不等于一个可以发布的系统；一个可以发布的系统也不等于一个可以在高风险场景中不受约束使用的系统。

目录

第一部分 基础	1
第一章 什么使语言模型变大	2
1.1 研究对象	2
1.2 从基础模型到助手	3
1.3 本书路线	3
1.4 深入展开：规模、系统与评测	3
1.5 章节细节	4
1.5.1 研究对象	4
1.5.2 预测为什么成为通用接口	4
1.5.3 规模改变了研究问题	4
1.5.4 从基础模型到助手	5
1.5.5 推理使推理阶段成为训练的一部分	5
1.5.6 全书路线	5
1.6 关键术语、实现要点与练习	5
1.7 结构化检查表	6
1.7.1 规模轴检查表	6
第二章 Token、语料与训练信号	7
2.1 数据就是规格	7
2.2 分词是建模选择	7
2.3 预训练之外的信号	8
2.4 深入展开：数据管线不是预处理	8
2.5 章节细节	9
2.5.1 数据作为规格	9
2.5.2 分词作为建模选择	9
2.5.3 语料构建与混合	9

2.5.4	把文本打包成训练序列	9
2.5.5	预训练之外的训练信号	9
2.5.6	污染、评测与来源	9
2.6	关键术语、实现要点与练习	10
2.7	结构化检查表	10
2.7.1	数据管线报告清单	10

第二部分 架构、优化与系统 11

第三章 Transformer 机制 12

3.1	张量契约	12
3.2	缩放点积注意力	12
3.3	残差、归一化与前馈层	12
3.4	深入展开：从张量形状理解 Transformer	13
3.5	章节细节	13
3.5.1	张量契约	13
3.5.2	嵌入与位置	14
3.5.3	缩放点积注意力	14
3.5.4	Mask 与因果性	14
3.5.5	残差流与归一化	14
3.5.6	前馈块	14
3.5.7	调试不变量	14
3.6	关键术语、实现要点与练习	15
3.7	结构化检查表	15
3.7.1	Transformer 实现检查	15

第四章 从 Transformer 到 GPT 16

4.1	Decoder-only 契约	16
4.2	训练循环	16
4.3	从 logits 到文本	16
4.4	深入展开：GPT 是统一接口	17
4.5	章节细节	17
4.5.1	Decoder-only 契约	17
4.5.2	因果语言建模	17
4.5.3	最小 GPT Block 栈	18

4.5.4	Batching、Packing 与 Loss 曲线	18
4.5.5	从 Logits 到文本	18
4.5.6	高效注意力与生成	18
4.5.7	评测提醒	18
4.6	关键术语、实现要点与练习	18
4.7	结构化检查表	19
4.7.1	生成策略对照	19
第五章	LLaMA 类架构	20
5.1	现代 decoder 默认配置	20
5.2	KV Cache 与注意力变体	20
5.3	MoE 与替代序列模型	20
5.4	深入展开: LLaMA 类设计的组合意义	21
5.5	章节细节	21
5.5.1	什么使 Decoder 成为 LLaMA 类	21
5.5.2	RMSNorm 与 Pre-Normalization	22
5.5.3	SwiGLU 前馈网络	22
5.5.4	旋转位置嵌入	22
5.5.5	KV Cache 与注意力变体	22
5.5.6	Tokenizer 与词表契约	22
5.5.7	长上下文	22
5.5.8	MoE 变体	23
5.5.9	超越 LLaMA 类 Decoder	23
5.5.10	评测提醒	23
5.6	关键术语、实现要点与练习	23
第六章	优化与预训练	24
6.1	预训练问题	24
6.2	AdamW、学习率与稳定性	24
6.3	可复现训练记录	24
6.4	深入展开: 预训练是统计目标和工程配方的耦合	25
6.5	章节细节	25
6.5.1	预训练问题	25
6.5.2	目标记账与数据顺序	26
6.5.3	AdamW 与参数组	26

6.5.4	Schedule、Warmup 与 Batch Size	26
6.5.5	梯度裁剪与稳定性	26
6.5.6	Checkpoint 与可复现性	26
6.5.7	计算最优规划	26
6.5.8	监控、评测与停止规则	27
6.5.9	实现笔记	27
6.6	关键术语、实现要点与练习	27
6.7	结构化检查表	28
6.7.1	训练 dashboard	28
第七章	分布式训练系统	29
7.1	内存与并行	29
7.2	通信与吞吐	29
7.3	实现纪律	29
7.4	深入展开：分布式训练的核心是记账	30
7.5	章节细节	30
7.5.1	为什么训练系统是模型的一部分	30
7.5.2	内存记账	31
7.5.3	数据并行	31
7.5.4	ZeRO 与 FSDP	31
7.5.5	Tensor Parallelism	31
7.5.6	Pipeline Parallelism	31
7.5.7	Expert Parallelism	31
7.5.8	激活检查点与重算	31
7.5.9	精度与通信	32
7.5.10	吞吐记账	32
7.5.11	运行可靠性	32
7.5.12	实现笔记	32
7.6	关键术语、实现要点与练习	32
7.7	结构化检查表	33
7.7.1	并行策略诊断表	33
第八章	推理与服务	34
8.1	Prefill 与 Decode	34
8.2	KV Cache、批处理与调度	34

8.3	下一代推理系统	34
8.4	深入展开：推理服务是另一套系统工程	35
8.5	章节细节	35
8.5.1	服务不是 Evaluation Mode	35
8.5.2	Prefill 与 Decode	35
8.5.3	KV Cache	36
8.5.4	Batching 与调度	36
8.5.5	内存管理与准入控制	36
8.5.6	量化与压缩	36
8.5.7	投机与并行解码	36
8.5.8	注意力 Kernel 与长上下文	36
8.5.9	流式 API、取消与安全过滤	37
8.5.10	负载测试与成本记账	37
8.5.11	实现笔记	37
8.6	关键术语、实现要点与练习	37
8.7	结构化检查表	38
8.7.1	推理服务成本分解	38

第三部分 适配 39

第九章	监督指令微调	40
9.1	接口转换	40
9.2	合成数据	40
9.3	局限	41
9.4	深入展开：SFT 是接口训练，不是完整对齐	41
9.5	章节细节	42
9.5.1	接口转换	42
9.5.2	指令数据作为契约	42
9.5.3	合成指令数据	42
9.5.4	训练细节	42
9.5.5	拒答与安全数据	42
9.5.6	评测	42
9.5.7	模仿的局限	43
9.6	关键术语、实现要点与练习	43

9.7 结构化检查表	43
9.7.1 SFT 数据类型与风险	43
第十章 参数高效适配	44
10.1 Adapter、Prompt 与 LoRA	44
10.2 QLoRA 与量化训练	44
10.3 深入展开：参数高效不等于风险高效	45
10.4 章节细节	45
10.4.1 到底在让什么高效	45
10.4.2 Adapter 与 Prompt 家族	46
10.4.3 LoRA	46
10.4.4 QLoRA 与量化训练	46
10.4.5 选择 Rank、目标模块与超参数	46
10.4.6 系统成本与部署	46
10.4.7 适配模型评测	46
10.5 关键术语、实现要点与练习	47
10.6 结构化检查表	47
10.6.1 PEFT 报告模板	47
第十一章 领域与语言适配	48
11.1 适配是一种诊断	48
11.2 继续预训练与检索	48
11.3 领域评测	48
11.4 深入展开：领域适配先诊断再训练	49
11.5 章节细节	49
11.5.1 适配是一种诊断	49
11.5.2 语言适配	50
11.5.3 继续预训练	50
11.5.4 监督领域微调	50
11.5.5 结构化任务：NL2SQL	50
11.5.6 检索还是更新权重	50
11.5.7 领域安全与治理	50
11.5.8 分布偏移下的评测	51
11.6 关键术语、实现要点与练习	51
11.7 结构化检查表	51

11.7.1 领域适配决策表	51
--------------------------	----

第四部分 对齐、应用与评测 52

第十二章 检索、工具与 Agent 53

12.1 RAG 作为控制系统	53
12.2 上下文工程与记忆	53
12.3 工具使用	54
12.4 Agent 工作流	54
12.5 深入展开：RAG、上下文和工具是一个控制系统	54
12.6 章节细节	55
12.6.1 RAG 作为控制系统	55
12.6.2 索引与检索	55
12.6.3 重排与上下文构造	55
12.6.4 带证据生成	55
12.6.5 RAG 评测	55
12.6.6 上下文工程与记忆	55
12.6.7 Prompt Injection 与信任边界	56
12.6.8 工具使用	56
12.6.9 Agent 与工作流	56
12.6.10 系统成本	56
12.7 关键术语、实现要点与练习	56
12.8 结构化检查表	57
12.8.1 RAG 设计选择	57

第十三章 偏好学习与对齐 58

13.1 偏好数据	58
13.2 RLHF、PPO 与 DPO	58
13.3 DPO 之后的偏好目标	58
13.4 安全与 AI Feedback	59
13.5 深入展开：偏好学习优化的是代理目标	59
13.6 章节细节	59
13.6.1 对齐作为偏好建模	59
13.6.2 从标签到比较	60
13.6.3 采样候选回答	60

13.6.4 标注协议	60
13.6.5 Bradley-Terry 目标	60
13.6.6 校准与不确定性	60
13.6.7 过优化	60
13.6.8 带 PPO 的 RLHF	60
13.6.9 PPO 机制	61
13.6.10 系统成本	61
13.6.11 DPO	61
13.6.12 取舍	61
13.6.13 DPO 之后的偏好目标	61
13.6.14 安全、拒答与 AI Feedback	61
13.6.15 评测提醒	61
13.7 关键术语、实现要点与练习	62
13.8 结构化检查表	62
13.8.1 偏好学习失败诊断	62
第十四章 推理与测试时计算	63
14.1 推理是预算化计算	63
14.2 RLVR 与 GRPO	63
14.3 自适应测试时计算	63
14.4 深入展开：推理方法首先是预算分配方法	63
14.5 章节细节	64
14.5.1 推理作为预算化计算	64
14.5.2 Chain of Thought	64
14.5.3 Self-Consistency	64
14.5.4 分解与程序化推理	64
14.5.5 Sample-and-Rank	65
14.5.6 结果监督与过程监督	65
14.5.7 树搜索	65
14.5.8 可验证奖励强化学习	65
14.5.9 GRPO	65
14.5.10 推理时扩展	65
14.5.11 预算强制	65
14.5.12 计算最优分配	66
14.5.13 前沿推理系统	66

目录	xi
14.5.14 答案准确率不够	66
14.5.15 轨迹忠实性	66
14.5.16 推理边界测试	66
14.6 关键术语、实现要点与练习	66
14.7 结构化检查表	67
14.7.1 推理方法比较	67
第十五章 多模态与生成式基础模型	68
15.1 模态作为接口	68
15.2 统一理解与生成	68
15.3 视频、音频与 Omni 模型	69
15.4 生成模型目标	69
15.5 行动、具身与世界模型	70
15.6 多模态评测与安全	71
15.7 深入展开：多模态和生成模型改变接口边界	71
15.8 章节细节	72
15.8.1 模态作为接口	72
15.8.2 图文对齐	72
15.8.3 零样本分类	72
15.8.4 冻结编码器与投影层	72
15.8.5 Query Transformer 与 Token 压缩	73
15.8.6 Cross-Attention 与交错媒体	73
15.8.7 架构取舍	73
15.8.8 统一理解与生成	73
15.8.9 生成建模目标	73
15.8.10 自回归生成	73
15.8.11 扩散与 Rectified Flow	73
15.8.12 混合 AR-Diffusion 系统	74
15.8.13 行动、具身与世界模型	74
15.8.14 多模态指令微调	74
15.8.15 训练阶段	74
15.8.16 数据来源	74
15.8.17 带图像的 Chat Template	74
15.8.18 OCR、图表与文档	74
15.8.19 系统成本	75

15.8.20 视觉 Token 与 Prefill	75
15.8.21 视频与音频	75
15.8.22 Benchmark 家族	75
15.8.23 评测提醒	75
15.8.24 安全与治理	75
15.8.25 视觉 Prompt Injection	75
15.8.26 隐私与敏感推断	76
15.8.27 幻觉与 Grounding	76
15.9 关键术语、实现要点与练习	76
15.10 结构化检查表	77
15.10.1 多模态系统报告清单	77
第十六章 评测、安全与治理	78
16.1 评测是测量系统	78
16.2 人类评测与 LLM-as-Judge	79
16.3 治理	79
16.4 可解释性、遗忘与水印	79
16.5 深入展开：评测是发布证据	80
16.6 章节细节	81
16.6.1 评测作为测量系统	81
16.6.2 从问题到决策	81
16.6.3 测量模板	81
16.6.4 能力 Benchmark	81
16.6.5 pass@k 与选择效应	81
16.6.6 Benchmark 污染	81
16.6.7 Benchmark 饱和与 Gaming	81
16.6.8 长上下文、Agent 与生成评测	82
16.6.9 Rubric 与成对偏好	82
16.6.10 LLM-as-Judge	82
16.6.11 真实性与事实精度	82
16.6.12 校准与弃答	82
16.6.13 鲁棒性	82
16.6.14 政策分类	82
16.6.15 红队	83
16.6.16 文档	83

16.6.17 风险管理框架	83
16.6.18 可解释性、遗忘与水印	83
16.6.19 运行监控	83
16.6.20 部署限制	83
16.7 关键术语、实现要点与练习	83
16.8 结构化检查表	84
16.8.1 评测与治理发布清单	84
第十七章 研究前沿与实践路线	85
17.1 为什么前沿实践会改变课程	85
17.2 路线图范围	85
17.3 前沿扩展方向	86
17.4 读者路线图	86
17.5 深入展开：为什么结构保持 17 章	87
17.6 章节细节	87
17.6.1 为什么需要前沿章	87
17.6.2 覆盖了什么，还需要关注什么	87
17.6.3 从零实践作为研究方法	88
17.6.4 前沿架构问题	88
17.6.5 推理与自适应测试时计算	88
17.6.6 数据、评测与治理作为一条闭环	88
17.6.7 读者后续路线图	88
17.7 关键术语、实现要点与练习	88
17.8 前沿证据矩阵	89
附录 A 附录：记号、实验记录与术语	90
A.1 常用记号	90
A.2 实验记录清单	90
A.3 术语表	90

第一部分

基础

第一章 什么使语言模型变大

1.1 研究对象

大语言模型的基本对象仍然是条件概率分布。给定 token 序列 $x_1, \dots, x_T$ ，自回归模型把联合概率分解为

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}).$$

训练通常最小化下一个 token 的交叉熵。这个目标看起来简单，却把语言、代码、数学、工具调用、对话格式和多模态描述都压缩进同一个预测界面。

“大”不只表示参数多。参数规模、训练 token 数、数据多样性、上下文长度、推理预算、工具能力、服务吞吐和安全约束都会改变模型行为。一个参数更小但上下文更长、检索更强、推理预算更大的系统，可能在真实任务中比一个参数更多但系统设计较弱的模型更有用。

因此，“大模型”的规模不是一个单轴数字。总参数量说明容量，激活参数量说明每个 token 实际花费的计算，训练 token 数说明模型见过多少统计证据，数据来源说明这些证据是否合法、可靠并且可复现。上下文长度说明模型一次前向可以条件化多少信息，但 KV cache、注意力代价和位置泛化决定了这个长度能否经济地服务。推理预算说明模型在预训练之后还能通过采样、检索、工具调用、验证器和搜索花多少额外计算。任何只报告参数量的比较，都无法解释现代系统的真实能力。

本书把模型看成一个“条件计算系统”。这种说法比“一个神经网络”更准确，因为同一个 checkpoint 在不同 tokenizer、chat template、解码参数、检索器、工具权限和安全过滤器下会表现出不同系统行为。基础模型学习的是文本延续分布；助手系统还要定义谁在说话、哪些指令有更高优先级、哪些内容必须拒绝、哪些外部证据可以信任，以及哪些工具可以被调用。

1.2 从基础模型到助手

基础模型通过预训练学习广泛的语言和知识模式，但它并不天然知道应该如何作为用户助手响应。后训练阶段通过监督指令微调、偏好学习、安全数据和工具协议，把基础模型转化为面向用户的系统。InstructGPT 和后续 RLHF 工作使这一点变得清晰：可用性不是预训练困惑度的直接结果，而是接口、偏好和安全目标共同塑造的结果 [63, 7]。

近年来，OpenAI o1/o3、DeepSeek-R1、Qwen3 和 Kimi K2 等系统又把“推理时间”变成产品能力的一部分。推理不再只是模型内部能力，也包括采样、搜索、验证、工具调用和预算分配 [17, 91, 40]。

这也解释了为什么预训练损失、聊天主观评分和真实部署质量之间不存在简单等价关系。预训练损失下降，可能意味着模型更好地预测了训练分布；但如果训练数据包含评测污染、错误网页、过期知识或有害模式，下游行为可能并不更可信。聊天偏好评分上升，可能说明模型语气更符合标注者期待；但它也可能学会冗长、奉承、空泛免责声明或过度拒答。真实部署则还要面对延迟、成本、权限、日志、监控和事故响应。

1.3 本书路线

第一部分讨论建模对象、token 和数据。第二部分讨论 Transformer、GPT、LLaMA 类结构、优化、分布式训练和推理服务。第三部分讨论 SFT、LoRA、QLoRA、领域和语言适配。第四部分讨论 RAG、工具、agent、偏好学习、推理型训练、多模态与生成式基础模型、评测、安全和治理。最后一章把 CS336 式从零实现纪律与 2025–2026 年前沿研究连接起来 [78]。

因此，本书虽然以大语言模型为主线，但并不把研究边界限死在纯文本模型。文本模型提供了最清晰的训练目标、系统接口和后训练范式；多模态理解模型、统一理解-生成模型、扩散/流匹配生成模型和 Omni 模型则说明，同一套数据、系统、评测和治理问题正在扩展到图像、音频、视频、语音和交互式生成。后文讨论这些模型时，会尽量避免把它们写成模型名称列表，而是把它们放回目标函数、表示、训练、服务和评测的共同框架中。

1.4 深入展开：规模、系统与评测

本章的重点不是给“大模型”下一个参数量阈值，而是拆解“规模”这个词背后的多个轴。参数量只是容量的一种表示；训练 token 数决定模型见过多少统计证据；数据来源决定这些证据是否可靠、合法、可复现；上下文长度决定推理时能条件化多少信息；推理预算决定模型能否在预训练之后通过采样、检索、验证、工具和搜索继续花计算。一个只报告“多少 B 参数”的模型比较，通常遮蔽了真正的系统差异。

这也是为什么本书从一开始就把大语言模型写成系统，而不是单个神经网络。模型权重、tokenizer、chat template、检索器、工具运行时、安全过滤器和服务调度器共同定义用户实际接触到的行为。两个团队使用同一 base checkpoint，如果一个团队有更好的检索、上下文构造、引用约束和安全回归测试，另一个团队只有普通聊天模板，它们在真实任务中的可靠性会完全不同。

本章还强调，预训练的“基础模型”并不会自动成为助手。基础模型学到的是延续文本的能力，而助手需要知道谁在说话、哪些指令优先级更高、什么时候需要拒答、什么时候应引用证据、什么时候应调用工具。SFT、RLHF、DPO、RLAIF 和推理型强化学习都是把基础模型转化为可交互系统的后训练机制，但它们优化的都是代理目标，不能被误解为“已经对齐人类价值”。

最后，评测从第一章就被放进主线。模型越大，越容易在公开 benchmark 上出现污染、调参和过拟合；上下文越长，越容易让人误以为模型真正利用了所有证据；推理采样越多，越容易用 pass@k 掩盖单次可靠性不足。因此，本书后续每章都反复追问同一个问题：某个指标到底测量了什么，遗漏了什么，是否足以支撑发布决策。

1.5 章节细节

1.5.1 研究对象

语言模型研究的对象首先是一个条件分布，而不是一个聊天界面。自回归分解把长文本生成化为一串局部预测，但这些局部预测在规模足够大、数据足够多、接口足够通用时，会表现为翻译、问答、代码、推理和工具调用等能力。理解这一点可以避免把每个能力都误读为单独训练出来的模块。

1.5.2 预测为什么成为通用接口

下一个 token 预测之所以重要，是因为它能容纳几乎所有可被序列化的任务。任务说明、输入材料、推理草稿、函数调用、程序输出和最终答案都能被写成上下文的一部分。它的代价是边界必须由数据格式和模板显式定义，否则模型会混淆角色、证据、指令和输出。

1.5.3 规模改变了研究问题

规模让研究从“模型是否能做某类任务”转向“系统如何稳定、经济、可解释地做任务”。参数、token、上下文、数据质量、后训练和服务预算共同决定能力。只讨论参数量会忽略训练 token 不足、推理成本过高、上下文利用率低和评测污染等关键问题。

1.5.4 从基础模型到助手

基础模型只是学会了延续文本，并不天然知道怎样服务用户。助手需要学习对话角色、拒答边界、格式约束、工具协议和安全策略。SFT、偏好学习和安全训练的意义正是在这个接口层面重塑模型行为。

1.5.5 推理使推理阶段成为训练的一部分

推理型模型把测试时计算纳入能力定义。模型可以生成多个候选、调用工具、使用验证器、搜索解空间或按难度分配 token。这样一来，比较模型时不能只问 checkpoint 有多强，还要问系统允许它在推理时花多少计算、用哪些外部资源、怎样选择答案。

1.5.6 全书路线

本书的路线从概率建模和数据开始，再进入 Transformer、GPT、LLaMA 类结构、预训练、分布式训练和推理服务，随后讨论指令微调、PEFT、领域适配、RAG、偏好学习、推理、多模态以及治理。这个顺序反映了一个判断：现代大模型不是单点算法，而是从数据到部署的一条完整链路。

1.6 关键术语、实现要点与练习

关键术语。条件语言模型指在给定前文时预测下一个 token 的概率模型；基础模型指经过大规模预训练但尚未面向助手接口后训练的模型；后训练包括 SFT、偏好优化、安全调优、推理强化和领域适配；推理预算指检索、采样、验证、搜索和工具调用在部署时消耗的额外计算；系统边界指模型权重之外的 tokenizer、模板、检索、工具、过滤器和服务运行时。

实现要点。读者应能写出一个最小自回归损失，并解释为什么 label 需要右移；能区分参数量、激活参数量、训练 token 数、上下文长度和推理预算；能为一个模型报告列出必须包含的数据、架构、训练、服务和评测字段。

练习。

1. 给定一个四 token 序列，写出自回归分解，并说明每一步条件是什么。
2. 比较两个系统：一个模型更大但无检索，另一个模型较小但有高质量 RAG。列出至少五个不能只看参数量的原因。
3. 为一个助手系统画出边界图：模型、tokenizer、模板、检索器、工具、安全过滤和日志分别在哪里。

- 4. 找一篇模型技术报告，标注其中哪些数字是能力数字，哪些是系统成本数字，哪些是治理证据。

1.7 结构化检查表

1.7.1 规模轴检查表

规模轴	它实际改变什么	常见误读
总参数量	模型容量、存储、加载成本	参数越多一定越强
激活参数量	每个 token 实际计算量	MoE 总参数可直接和 dense 比
训练 token 数	统计证据和覆盖面	token 多就一定数据好
上下文长度	推理时可条件化的信息量	能放入就等于能利用
推理预算	采样、搜索、检索、验证、工具成本	多想一定更正确
服务吞吐	用户可用性和成本	离线 benchmark 足够代表部署

第二章 Token、语料与训练信号

2.1 数据就是规格

数据不是模型之外的附件。它定义了模型可见的语言、知识、风格、许可证边界和风险。训练语料中的重复内容会增加记忆风险；低质量网页会教会模型无意义模板；代码数据会改变推理和格式能力；多语言数据会影响 token 效率和跨语言泛化。

一个严肃的数据报告至少应记录来源、时间、许可证、语言分布、过滤规则、去重方法、敏感信息处理、合成数据生成器、评测集重叠检查和数据混合权重。没有这些信息，模型行为很难解释，评测结论也很难复现。

数据混合权重本身就是训练目标的一部分。把更多 token 分配给代码，会提升代码补全、格式遵循和某些结构化推理能力，但也会改变自然语言风格和安全风险。把更多 token 分配给数学和竞赛题，可能提升推理 benchmark，却也可能让模型在普通对话中更爱展开冗长步骤。多语言数据不仅影响语言覆盖，还影响 token fertility：同样的中文、阿拉伯文或印地语内容，如果被 tokenizer 切得更碎，就会占用更多训练和推理计算。

过滤不是单纯“清洗脏数据”。它会改变模型世界观和能力边界。过强过滤可能删掉少数语言、非正式文本、敏感但合法的讨论和真实错误案例；过弱过滤则会保留垃圾模板、个人信息、仇恨内容、恶意代码和重复材料。高水平数据管线应把过滤规则视为可检查决策，而不是一次性脚本。

2.2 分词是建模选择

Tokenizer 把文本转成离散 token。BPE、SentencePiece 和 byte fallback 等方案不只是预处理工具，它们改变序列长度、词表大小、嵌入矩阵形状和模型可学习的单位 [74, 42]。对中文、代码、数学符号和低资源语言而言，token fertility 尤其重要：同样一句话被切成更多 token，会占用更多上下文和训练计算。

Tokenizer 与模型权重绑定。更换 tokenizer 会改变 token id、特殊 token、聊天模板和长度分布。生产系统应把 tokenizer、chat template、special tokens 和模型权重一起版本化。

2.3 预训练之外的信号

监督指令数据把结构化对话序列化为 token；偏好数据把一个 prompt 下的候选回答排序；验证器数据用单元测试、数学答案或工具执行结果给出奖励；检索语料在推理时提供外部证据。它们都不是简单文本，必须记录格式、掩码、边界和评测污染风险。

评测污染是现代大语言模型研究的核心问题。模型可能见过原题、改写、答案解析、翻译版本或合成变体。任何高分都应回答一个问题：这个分数测量的是泛化能力，还是训练和开发流程已经接触过测试信息 [88]？

去重同样有多层含义。训练集内部去重可以避免重复文档被过度加权；训练集和评测集之间去重可以保护 benchmark 有效性；跨许可证去重可以避免禁止来源通过镜像或转载重新进入语料。近重复比精确重复更难处理，因为一个答案解析、论坛转载或自动翻译版本都可能携带测试信息。对于高价值评测，团队应同时做字符串级、token n-gram、文档来源和语义模板级检查。

2.4 深入展开：数据管线不是预处理

本章把数据称为“规格”，意思是数据管线不是训练前的杂务，而是决定模型行为的第一层工程。一个语料混合会显式或隐式规定模型应该擅长哪些语言、哪些知识领域、哪些文本风格、哪些代码习惯和哪些安全边界。网页、书籍、论文、代码、问答、论坛、对话、数学题和工具轨迹都携带不同信号；把它们混在一起之前，必须知道每类数据的来源、时间、许可证、清洗规则和权重。

Tokenizer 在本章中被视为建模选择，而不是无害工具。词表大小影响 embedding 和输出矩阵；切分规则影响序列长度；byte fallback 影响罕见字符和多语言覆盖；special token 影响聊天角色、工具调用和多模态占位符。对中文而言，同一句话如果被切成更多 token，就会在训练和推理中花费更多步数；对代码而言，缩进、括号、标识符和空白的切分会影响生成格式稳定性。换 tokenizer 等同于改变模型输入语言，不能在预训练后随意替换。

本章还区分了多种训练信号。预训练数据是 continuation signal，SFT 数据是 demonstration signal，偏好数据是 comparison signal，RLVR 数据是 verification signal，RAG 语料是 inference-time evidence。它们的共同点是都必须被序列化为模型可处理的 token 或上下文，但它们的语义完全不同。把偏好数据当作普通文本训练，会丢掉 chosen/rejected 关系；把工具轨迹当作普通对话训练，会丢掉 schema、权限和错误通道。

污染控制是这一章的另一个中心。本章强调污染不是二元问题。模型可能见过原题、答案、解析、相似模板、翻译版本、合成变体或论坛讨论。越是被频繁用于模型选择的公开 benchmark，越需要像科学实验中的 test set 一样保护。严肃团队应在数据进入训练前建立 manifest，并在发布报告中说明污染检查方法，而不是只写一句“we deduplicated the data”。

2.5 章节细节

2.5.1 数据作为规格

数据决定模型能看见什么、不能看见什么、反复看见什么，以及以什么风格看见。语料来源、时间、许可证、重复率、语言分布、代码比例和安全过滤都会进入最终行为。把数据称为规格，是因为它像 API 合同一样定义系统边界。

2.5.2 分词作为建模选择

Tokenizer 决定文本被切成哪些基本单位。它影响序列长度、上下文消耗、嵌入矩阵大小、特殊 token 设计和多语言公平性。中文、代码、数学符号和低资源语言尤其容易受到 token fertility 的影响，因此 tokenizer 不能被视为无关预处理。

2.5.3 语料构建与混合

语料混合不是把更多文件堆在一起，而是决定不同数据源的训练权重。网页、书籍、论文、代码、论坛、对话和合成数据各自带来不同能力和风险。好的数据报告应说明每类来源如何过滤、去重、抽样和加权。

2.5.4 把文本打包成训练序列

Packing 提高吞吐，但也容易制造边界错误。多个文档被放进同一序列时，模型不应跨文档泄漏信息；padding、attention mask、position id 和 loss mask 必须一致。很多看似神秘的训练异常，实际来自序列边界和标签右移错误。

2.5.5 预训练之外的训练信号

现代模型使用的不只是 continuation signal。指令数据提供示范，偏好数据提供比较，验证器数据提供可执行奖励，检索语料在推理时提供证据。每类信号都有自己的格式、掩码和风险，不能简单当作普通文本混入。

2.5.6 污染、评测与来源

评测污染包括原题、答案、解析、翻译、改写和合成变体。模型越强、语料越大，越难保证公开 benchmark 未被接触。来源记录、近重复检测和时间切分不是形式主义，而是让评测仍然有解释力的前提。

2.6 关键术语、实现要点与练习

关键术语。 Token fertility 表示同一文本被切成 token 的密度；data manifest 记录数据来源、时间、许可证和过滤规则；去重包括训练集内部、训练-评测之间和跨许可证去重；评测污染指训练或调参过程接触到测试信息；label mask 表示哪些 token 参与损失。

实现要点。 数据管线应输出可复现 manifest；tokenizer、special tokens 和 chat template 必须版本化；SFT、偏好、检索和评测数据应保持不同 schema；污染检查应包含 exact match、n-gram、来源和语义模板检查。

练习。

1. 选取中文、英文、代码和数学各三句话，比较同一 tokenizer 的 token fertility。
2. 设计一个数据 manifest 表格，包含来源、许可证、时间、语言、过滤规则和权重。
3. 解释为什么“删除完全重复”不足以防止 benchmark contamination。
4. 写出一个 SFT 样本的 label mask，并说明为什么用户 token 不应计算 assistant loss。

2.7 结构化检查表

2.7.1 数据管线报告清单

字段	应记录内容
来源	域名、数据集名、采集时间、镜像关系、是否合成
许可证	训练、再分发、商业使用、撤回和检查要求
语言与领域	语言分布、代码/数学/网页/论文/对话比例
过滤	质量过滤、PII 处理、安全过滤、低质模板删除
去重	exact、near-duplicate、跨 split、跨许可证去重
污染检查	benchmark 原题、答案、解析、翻译和合成变体重叠
混合权重	各来源采样概率、温度采样、epoch 或 token 上限

第二部分

架构、优化与系统

第三章 Transformer 机制

3.1 张量契约

Transformer 的实现首先是张量契约。输入通常具有形状 $B \times T$ ，经过嵌入层变成 $B \times T \times d$ 。注意力层、前馈层、残差连接和归一化层都必须保持形状一致。许多训练错误不是数学思想错误，而是 mask、padding、batch 维度或 dtype 处理错误。

理解 Transformer 时，最可靠的方法不是背图，而是追踪张量。一个位置在每层都保留隐藏向量，注意力子层决定不同位置如何交换信息，MLP 子层决定每个位置内部如何变换，残差流把这些变换累积起来。只要某个实现无法清楚说明 batch 维、序列维、head 维和 hidden 维如何变化，它就很难被调试。

3.2 缩放点积注意力

对查询 Q 、键 K 、值 V ，注意力计算为

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} + M \right) V.$$

mask M 决定 token 能看见什么。因果语言模型使用上三角 mask，保证第 t 个位置不能看到未来 token。这个 mask 是自回归训练和生成的根本约束。

mask 约定是工程上最容易出错的部分之一。某些库用布尔值 True 表示允许注意，另一些库用 True 表示屏蔽；加性 mask 用一个极小负数近似 $-\infty$ ，乘性 mask 则改变概率归一化前后的语义。混合精度下，如果 softmax 前没有减去行最大值，或者 masked position 的数值不够小，就可能产生 NaN、未来信息泄漏或训练损失异常偏低。

3.3 残差、归一化与前馈层

现代 decoder 通常使用 pre-norm 结构，把归一化放在子层之前，以改善深层训练稳定性。前馈层对每个 token 独立作用，提供非线性容量。SwiGLU 等门控前馈结构在 LLaMA 类模型中很常见 [75]。

注意力权重可以辅助诊断，但不能被当作完整解释。一个 head 关注某个 token，并不证明该 token 导致了最终输出；value projection、后续层、MLP 和残差路径都可能改变或擦除信息。因此，分析 Transformer 时应结合注意力可视化、受控干预、held-out loss、下游任务和生成行为。

3.4 深入展开：从张量形状理解 Transformer

本章的写法是“从张量契约推导结构”。输入 token id 的形状通常是 $B \times T$ ，embedding 后变成 $B \times T \times d$ 。多头注意力再把隐藏维度拆成 head 数和每个 head 的维度，形成 query、key、value。只要这些形状的含义不清楚，模型实现就会在 mask、padding、cache 或并行切分处出错。

缩放点积注意力中的 $\sqrt{d_h}$ 不是装饰项。head 维度变大时，未缩放的点积方差会增长，softmax 容易过早变尖，梯度信号变差。softmax 的数值实现也必须稳定：通常先减去每行最大值，再加入 mask，并在混合精度中谨慎处理极小负数。许多“模型不收敛”的问题，实际是 attention score overflow、mask 方向写反或 padding 被模型看见。

因果 mask 是语言模型目标的一部分。如果训练时第 t 个位置能看到未来 token，训练 loss 会异常好看，但生成时模型无法访问未来信息，结果会崩坏。packed sequence、left padding、sliding window 和 KV cache 会让“数组位置”和“逻辑位置”不同，因此 position ids 应被显式构造和测试。长上下文扩展时，位置处理错误会在短样本上看不出来，却在长文档中造成严重退化。

残差流是理解深层 Transformer 的核心。每层 attention 和 MLP 都不是替换隐藏状态，而是在残差路径上增量写入信息。Pre-norm 让子层输入分布更稳定，也让梯度更容易穿过深层网络。Attention heads、MLP neurons、残差通道和 normalization 共同塑造行为，所以解释模型时不能把某个 attention map 直接等同于“模型理由”。

3.5 章节细节

3.5.1 张量契约

Transformer 的核心首先是形状不变量。输入 token id、embedding、query、key、value、attention logits、head 合并和输出投影都有明确维度。实现者如果不能逐步说清这些维度，就很难发现 mask、cache 和并行切分中的错误。

3.5.2 嵌入与位置

Token embedding 把离散 id 映射为向量，位置机制让模型知道序列顺序。绝对位置、相对位置和 RoPE 等方案把顺序信息注入模型的方式不同。长上下文扩展时，位置外推和训练长度分布会直接影响模型能否真正利用远距离信息。

3.5.3 缩放点积注意力

注意力用 query 与 key 的相似度选择 value 的加权组合。除以 $\sqrt{d_k}$ 是为了控制点积分布尺度，避免 softmax 过早饱和。数值稳定实现需要处理 mask、极小值、混合精度和行最大值，否则训练可能在大模型上失控。

3.5.4 Mask 与因果性

因果 mask 定义了语言模型的任务：当前位置只能看见过去。Padding mask、防泄漏 mask、packed sequence mask 和 sliding-window mask 都会改变可见信息。训练时一次 mask 写反，就可能得到虚假的低 loss 和不可用的生成模型。

3.5.5 残差流与归一化

残差连接让每层在已有表示上写入增量信息，而不是完全替换隐藏状态。Pre-norm 让梯度更容易穿过深层网络，RMSNorm 和 LayerNorm 则控制激活尺度。解释模型时应理解残差流、attention 和 MLP 的共同作用，而不是只看某个注意力图。

3.5.6 前馈块

MLP 或 FFN 是逐位置的非线性变换，通常占 Transformer 计算的大头。SwiGLU 等门控结构提高表达能力，但也改变参数量和激活分布。MoE 可以把 FFN 变成稀疏专家集合，从而扩大总容量但增加路由和通信问题。

3.5.7 调试不变量

小模型调试应检查形状、mask、loss 右移、梯度、初始 loss、过拟合小 batch 和生成样例。一个能在小数据上收敛并复现实验不变量的实现，才值得扩展到昂贵训练。CS336 式训练纪律的核心就是先让每个局部假设可测。

3.6 关键术语、实现要点与练习

关键术语。 Residual stream 是贯穿所有层的隐藏状态；causal mask 阻止当前位置看到未来 token；multi-head attention 让模型在多个投影子空间中比较 token；pre-norm 把归一化放在子层之前；attention weights 是诊断信号但不是完整解释。

实现要点。 每个实现都应测试张量形状、mask 方向、padding 处理和 position ids；softmax 前应做数值稳定处理；packed sequence 和 KV cache 下逻辑位置可能不等于数组位置；解释模型行为时应结合干预而不是只看注意力图。

练习。

- 1. 给定 $B = 2, T = 4, d = 8, h = 2$ ，写出 Q, K, V 和 attention score 的形状。
- 2. 写一个因果 mask 矩阵，并说明 additive mask 和 boolean mask 的差异。
- 3. 解释 pre-norm 为什么通常比 post-norm 更容易训练深层 decoder。
- 4. 设计一个测试，检测模型是否在训练时偷看未来 token。

3.7 结构化检查表

3.7.1 Transformer 实现检查

组件	必须验证	典型失败
Embedding	token id、padding、special token	特殊 token 与模板不一致
Attention mask	causal、padding、packed sequence	偷看未来或只看 padding
Position ids	left padding、cache、长上下文	位置漂移导致长文档退化
Softmax	行最大值、mask 数值、dtype	overflow、NaN、异常低 loss
Residual/norm	pre-norm 顺序、残差形状	深层训练不稳定
KV cache	cache append、位置更新、释放	生成错位或显存泄漏

第四章 从 Transformer 到 GPT

4.1 Decoder-only 契约

GPT 类模型去掉 encoder-decoder 结构，只保留因果 decoder。它的统一接口是：给定前文，预测下一个 token。问答、翻译、代码补全、函数调用和对话都可以被写成同一个序列建模问题。

这种统一接口的优势是简单和可扩展。模型不需要为每个任务定义新的输出头，只要把任务、上下文、角色和目标格式写进 token 序列即可。代价是所有边界都必须被序列化：system message、user message、assistant answer、工具调用、工具返回、图像占位符、结束符和拒答格式都必须变成模型能学习的 token 模式。

4.2 训练循环

一个最小 GPT 训练循环包括：取样 token block，前向计算 logits，右移标签，计算交叉熵，反向传播，梯度裁剪，优化器更新，周期性验证和 checkpoint。损失曲线应按数据源、长度和任务切片观察，而不是只看全局平均。

训练循环的很多错误只在细节中出现。标签右移错一位会让模型预测当前 token 而不是下一个 token；padding token 没有 mask 会让模型学习填充模式；gradient accumulation 没有正确缩放 loss 会改变有效学习率；checkpoint 没有保存 optimizer state 会使恢复训练后的曲线出现跳变。小模型调试阶段应该专门写测试检查这些不变量。

4.3 从 logits 到文本

生成时，模型把 logits 转成分布。温度控制分布尖锐程度；top- k 和 top- p 控制候选集合；重复惩罚影响循环文本；stop token 和 chat template 控制响应边界。解码策略不是无关细节，它直接影响事实性、创造性、长度、成本和安全过滤。

结构化输出使解码更复杂。JSON、SQL、函数调用和工具参数都要求模型在自然语言

之外遵守语法约束。生产系统通常需要增量解析、schema validation、失败修复和安全沙盒。一个模型在自由文本中表现良好，并不代表它能稳定产生可执行、可解析、可授权的结构化动作。

4.4 深入展开：GPT 是统一接口

本章把 GPT 类模型解释为一种极简但强大的接口：只保留因果 decoder，让所有任务都变成“给定前缀，预测下一个 token”。翻译、问答、摘要、代码补全、函数调用、对话和推理轨迹都可以被写成序列。这个统一接口极大降低了任务工程复杂度，也使模型能从不同任务的序列模式中迁移。

但统一接口也把很多责任推给数据格式。系统消息和用户消息要有明确边界；assistant 的回答要有终止符；工具调用要能和自然语言区分；多轮对话要避免角色混淆；多模态输入要有图像、音频或视频占位符；结构化输出要能被解析器验证。本章强调，chat template 不是 UI 文案，而是训练和推理共同依赖的协议。

训练循环看似简单，实际上有许多不变量。标签必须右移，loss mask 必须只覆盖应预测的 token，gradient accumulation 必须正确缩放，clip 应在 unscale 之后执行，scheduler 应和 optimizer step 对齐，checkpoint 应包含 optimizer 和随机状态。小模型实现练习的价值就在这里：它让读者在规模变大前看到这些错误。

从 logits 到文本的最后一步同样是建模。贪心解码确定但容易僵硬；温度提高多样性但增加错误；top- p 截断尾部但会改变罕见事实的出现概率；重复惩罚可能减少循环，也可能破坏代码或数学符号；stop sequence 如果按文本而非 token 处理，可能截断过早或过晚。服务级生成循环必须把采样、停止、流式输出、结构验证和取消机制放在同一个状态机里。

4.5 章节细节

4.5.1 Decoder-only 契约

GPT 类模型保留因果 decoder，把任务统一成前缀条件下的下一个 token 预测。它没有任务专用输出头，所有任务边界都通过序列化格式表达。这个契约简单、可扩展，但也让模板和特殊 token 变得极其重要。

4.5.2 因果语言建模

训练时，模型看到 $x_{<t}$ 并预测 x_t 。标签右移、loss mask 和 padding 处理必须正确；否则模型可能学会预测当前 token 或填充 token。因果语言建模的简洁性掩盖了大量实现细

节。

4.5.3 最小 GPT Block 栈

一个最小 GPT 包括 token embedding、位置机制、若干 decoder block、最终归一化和输出头。每个 block 通常由 attention、MLP、残差和归一化组成。实现时应确认参数初始化、权重 tying、dropout、dtype 和 checkpoint 保存都与训练目标一致。

4.5.4 Batching、Packing 与 Loss 曲线

Batching 决定吞吐，packing 决定 token 利用率，loss 曲线决定训练健康。全局平均 loss 往往掩盖不同数据源、长度和语言上的差异。应按切片观察验证 loss，并用生成样例检查模型是否只是学会了格式。

4.5.5 从 Logits 到文本

Logits 经过温度、top- k 、top- p 、重复惩罚和停止规则才变成文本。采样策略直接影响创造性、事实性、循环、长度和安全性。结构化输出还需要解析、schema 检查、失败重试和工具权限控制。

4.5.6 高效注意力与生成

FlashAttention、KV cache、GQA/MQA 和连续 batching 让大模型生成可服务化。训练和推理的计算形态不同：训练并行处理序列，推理逐步扩展前缀。理解这种差别是分析延迟、显存和吞吐的基础。

4.5.7 评测提醒

GPT 能生成流畅文本不等于它可靠。评测应区分格式遵循、事实正确、推理正确、代码可执行、安全边界和长上下文利用。尤其在开放式任务中，主观偏好分数不能替代可复现的错误分析。

4.6 关键术语、实现要点与练习

关键术语。 Decoder-only model 只用因果 decoder 建模前缀到下一个 token；chat template 定义系统、用户、助手和工具消息的序列化方式；sampling policy 包括 temperature、

top-*p*、top-*k*、stop sequence 和 repetition penalty；structured output 要求模型输出满足 schema。

实现要点。最小训练循环必须验证右移标签、loss mask、gradient accumulation、checkpoint 和采样；服务级生成循环应分离请求状态和模型执行；结构化输出需要增量解析、schema 验证和失败恢复。

练习。

- 1. 把一个三轮对话序列化成 chat template，并标出哪些 token 计算 loss。
- 2. 比较 greedy、temperature sampling 和 top-*p* sampling 的失败模式。
- 3. 设计一个 JSON 输出 schema，并说明模型输出不合法时系统应如何处理。
- 4. 写出一个最小 generation loop 的状态字段：token ids、stop 条件、采样参数、stream handle 和取消标志。

4.7 结构化检查表

4.7.1 生成策略对照

策略	适用场景	风险
Greedy	需要确定性、低成本输出	僵硬、可能陷入局部高概率错误
Temperature	创意写作、多样回答	事实性和格式稳定性下降
Top- <i>p</i>	控制长尾采样	截断可能丢掉罕见正确项
Self-consistency	数学、推理、多路径任务	成本上升, pass@ <i>k</i> 掩盖 pass@1
Verifier selection	代码、数学、可检查任务	验证器漏洞和选择偏差
Structured decoding	JSON、SQL、工具调用	解析失败、schema 漏洞、修复循环

第五章 LLaMA 类架构

5.1 现代 decoder 默认配置

LLaMA 类模型通常使用 pre-norm、RMSNorm、RoPE、SwiGLU、无 bias 线性层、grouped-query attention、长上下文和更复杂的数据混合 [82, 83, 25]。这些选择的价值不是单独存在的，而是在训练稳定性、推理成本和后训练兼容性之间形成组合。

RoPE 把位置信息写进 query/key 的旋转结构，使相对位置关系更自然地进入注意力分数。RMSNorm 用均方根尺度归一化隐藏状态，减少均值项处理并常用于大规模 decoder。SwiGLU 通过门控提升前馈层表达能力。GQA/MQA 减少 KV cache，但也改变注意力头共享方式。教材应该把这些组件解释为服务于稳定性、长上下文和推理成本的组合，而不是模型报告中的装饰词。

5.2 KV Cache 与注意力变体

推理时，已生成 token 的 key/value 可以缓存，避免每步重复计算全部前缀。KV cache 大小随层数、头数、维度、上下文长度和 batch 增长。GQA/MQA 通过共享 key/value 头减少缓存成本，但可能改变质量和长上下文行为 [1]。

长上下文并不只增加注意力计算，它还增加服务系统中的状态。一个请求生成到第 10,000 个 token 时，服务器需要保留大量 KV block；如果用户取消、超时或切换工具，系统还要释放或复用这些 block。架构选择和调度策略在这里相互耦合：同一个模型结构，在不同 batch、不同上下文长度和不同请求混合下，成本曲线可能完全不同。

5.3 MoE 与替代序列模型

MoE 通过稀疏激活增加总容量，每个 token 只选择部分专家。它改变了“模型大小”的含义：必须区分总参数、激活参数、路由成本和专家通信。DeepSeek-V3 与 Kimi K2 显示了 MoE 在开放权重前沿系统中的重要性 [18, 40]。

RetNet、Mamba、Mamba-2 和 Titans 等工作探索了注意力之外的长序列机制 [80, 26, 16, 9]。但替代注意力的主张必须在相同数据、tokenizer、训练预算、上下文任务和服务负载下验证。

MoE 报告尤其要避免误导。总参数大不等于每 token 计算大，激活参数小也不等于服务便宜，因为专家路由、all-to-all 通信、负载均衡和冷专家问题都会影响吞吐。替代序列模型同样不能只报告困惑度；它们必须证明自己在检索、聚合、矛盾处理、后训练、工具调用和安全评测中仍然可靠。

5.4 深入展开：LLaMA 类设计的组合意义

本章并不把 LLaMA 类架构写成组件清单，而是解释这些组件为什么经常一起出现。具体说，RMSNorm 简化归一化，RoPE 写入相对位置信息，SwiGLU 提升前馈层表达。GQA 与 MQA 主要降低 KV cache 成本；无 bias 线性层和 pre-norm 则服务于训练稳定性。这些选择单独看都不神秘，组合在一起才形成现代 decoder 默认配方。

RoPE 和长上下文尤其需要谨慎。扩大 context window 不是简单提高一个超参数。位置编码外推、attention score 范围、训练长度分布、KV cache 显存和 RAG 上下文构造都会影响长文档表现。一个模型能接受长输入，并不代表它能稳定利用中间证据、跨章节聚合信息或抵抗干扰文档。

KV cache 是架构和服务交汇处。训练时所有 token 并行计算；生成时每一步只新增一个 token，但要保留历史 key/value。GQA/MQA 通过共享 key/value 头减少缓存，但可能影响模型质量和长上下文行为。服务系统中的 batch、分页缓存、请求取消和多租户隔离，都依赖这些结构选择。

MoE 则改变“模型规模”的语言。一个 MoE 模型可以有巨大的总参数量，但每个 token 只激活部分专家。质量、成本和延迟取决于 router、top-k 专家、负载均衡、专家并行和通信拓扑。因此要求读者看到模型报告中的总参数、激活参数和硬件条件，而不是被单一数字误导。

5.5 章节细节

5.5.1 什么使 Decoder 成为 LLaMA 类

LLaMA 类模型通常组合 pre-norm、RMSNorm、RoPE、SwiGLU、GQA、无 bias 线性层和高质量数据配方。它不是一个单一组件，而是一套现代 decoder 工程默认值。理解这种组合比记住某个模型名更重要。

5.5.2 RMSNorm 与 Pre-Normalization

Pre-norm 在子层前归一化输入，使深层训练更稳定。RMSNorm 使用均方根尺度，不显式减均值，常用于大规模 decoder。归一化选择会影响梯度流、数值稳定和后训练兼容性。

5.5.3 SwiGLU 前馈网络

SwiGLU 用门控结构增强前馈层表达能力。相比普通 ReLU 或 GELU FFN，它通常带来更好的质量-计算权衡。实现时需要注意 hidden dimension、参数量和初始化，因为 FFN 往往是计算和参数的主要来源。

5.5.4 旋转位置嵌入

RoPE 通过旋转 query 和 key 把相对位置信息写入注意力分数。它与长上下文扩展、频率缩放和位置外推密切相关。仅仅把最大长度调大，不等于模型真的学会了长距离检索和聚合。

5.5.5 KV Cache 与注意力变体

推理时缓存历史 key/value 能避免重复计算前缀。GQA 和 MQA 通过共享 key/value 头减少 cache 体积，但可能改变质量和长上下文行为。服务报告应说明 cache 成本、上下文长度、batch 形态和注意力实现。

5.5.6 Tokenizer 与词表契约

LLaMA 类 checkpoint 与 tokenizer、special tokens 和 chat template 绑定。更换 tokenizer 会改变输入 id 和输出空间，通常等同于换了模型接口。多语言和代码任务尤其依赖词表覆盖与切分效率。

5.5.7 长上下文

长上下文能力包括可接受长输入、在长输入中定位证据、跨段落整合信息和抵抗无关干扰。很多模型报告只证明了第一点。严肃评测应包括 needle、multi-hop、矛盾证据、长文档摘要和真实 RAG 工作负载。

5.5.8 MoE 变体

MoE 用稀疏专家扩大总参数量，每个 token 只激活部分专家。它带来容量优势，也带来路由、负载均衡、all-to-all 通信和专家退化问题。报告 MoE 时应区分总参数、激活参数、训练 FLOPs 和服务延迟。

5.5.9 超越 LLaMA 类 Decoder

RetNet、Mamba、Mamba-2、Titans 和扩散语言模型等工作探索注意力之外的序列建模。它们提出不同的长序列和生成机制，但必须在相同数据、预算、上下文任务和后训练流程下比较。单一困惑度不能证明替代结构已解决真实系统问题。

5.5.10 评测提醒

架构比较最容易被不公平设置污染。数据、tokenizer、训练 token、上下文长度、推理预算和后训练都可能比结构本身更影响结果。评测应固定关键变量，或者明确承认比较的是完整配方而非单一架构。

5.6 关键术语、实现要点与练习

关键术语。 RoPE 用旋转方式编码位置信息；RMSNorm 用均方根归一化隐藏状态；SwiGLU 是门控前馈结构；GQA/MQA 通过共享 key/value 头降低 KV cache；MoE 通过稀疏专家提升总容量。

实现要点。 报告架构时应写清层数、隐藏维、head 数、KV head 数、FFN 维度、位置编码、词表和上下文长度；MoE 报告必须区分总参数、激活参数、router、top-k 专家和专家并行；长上下文主张必须配合有效上下文评测。

练习。

1. 比较 MHA、MQA 和 GQA 的 KV cache 大小。
2. 解释为什么长上下文能力不能只看最大输入 token 数。
3. 给一个 MoE 模型报告写检查清单：哪些数字可能误导读者？
4. 比较 attention、state-space 和 recurrent memory 方案在长文档任务中的评测要求。

第六章 优化与预训练

6.1 预训练问题

预训练配方是架构、token 预算、数据混合、优化器、精度、batch、上下文长度、硬件拓扑和停止规则的耦合选择。Scaling laws 显示损失随参数、数据和计算可预测变化；Chinchilla 进一步说明许多早期大模型相对数据量而言参数过多 [37, 31]。

预训练损失必须按 token 解释。一个 batch 中不同数据源、不同长度和不同语言的 token 贡献不同，平均 loss 可能掩盖某个语料切片已经退化。训练计划还要区分 tokens seen、unique tokens、effective batch size、sequence length 和 gradient accumulation。两个运行即使总 FLOPs 接近，也可能因为数据顺序、warmup、上下文长度或验证集不同而不可比较。

6.2 AdamW、学习率与稳定性

AdamW 将权重衰减与自适应梯度更新解耦，是现代大语言模型训练的常见优化器 [52]。Warmup 防止早期更新过大；cosine decay 或类似 schedule 控制后期收敛；梯度裁剪和混合精度处理数值稳定性。BF16 常用于大模型训练，因为指数范围比 FP16 更稳。

稳定性不是单个超参数能保证的。pre-norm、初始化、残差尺度、激活函数、attention score 范围、MoE router、长上下文位置外推和低精度格式都会影响梯度分布。训练 dashboard 应区分优化健康和模型质量：前者包括 loss、gradient norm、update norm、learning rate、clipping rate、tokens/s、显存和通信时间；后者包括代码、数学、多语言、安全、事实性和生成样例。

6.3 可复现训练记录

训练报告应记录随机种子、数据 manifest、tokenizer、模型配置、optimizer state、精度、并行策略、checkpoint 频率、验证集、损失切片、硬件和软件版本。没有这些记录，一

个损失曲线只能说明“某次运行发生过”，不能支持科学解释。

checkpoint 不是简单保存权重。要可靠恢复训练，还需要优化器状态、scheduler 状态、随机数状态、数据加载位置、并行切分配置和 tokenizer/chat template 版本。大规模运行中，失败恢复本身就是训练系统的一部分：checkpoint 太频繁会浪费 I/O，太稀疏会增加故障成本；只让 rank 0 写日志也要保证所有 worker 在 barrier 和种子上保持一致。

6.4 深入展开：预训练是统计目标和工程配方的耦合

本章把预训练写成一个耦合系统。交叉熵目标定义了 token 级预测，optimizer 决定参数如何更新，data mixture 决定梯度来自哪里，batch size 和 sequence length 决定显存和统计效率，learning-rate schedule 决定训练早期和后期的稳定性，精度格式决定数值范围和吞吐。任何一个选择改变，都可能让“同样的模型”变成不同实验。

Scaling laws 的作用不是崇拜规模，而是帮助规划计算。Kaplan 风格 scaling law 显示 loss 随参数、数据和计算呈规律变化；Chinchilla 进一步说明许多早期模型相对训练 token 不足，参数过多而数据不够。实践中，团队需要在给定预算下决定参数量、token 数、训练步数和停止点，而不是只追求最大模型。

优化稳定性来自许多小机制的共同作用。AdamW 解耦权重衰减；warmup 防止早期大更新；cosine decay 控制后期学习率；gradient clipping 限制异常梯度；BF16 提供比 FP16 更大的指数范围；FP8 需要额外缩放和验证。MoE、长上下文和多模态输入都会改变激活和梯度分布，因此“默认超参数”不能脱离架构版本。

本章还强调训练 dashboard 应分成两类指标。优化健康看 training loss、validation loss、gradient norm、update norm、tokens/s、显存、通信时间和数据加载等待；模型质量看代码、数学、多语言、事实性、安全、长上下文和生成样例。一个训练运行可能在优化指标上很健康，却在某个能力切片上退化。只有把两类指标分开，才能判断问题来自训练系统还是数据和目标。

6.5 章节细节

6.5.1 预训练问题

预训练把模型结构、数据混合、token 预算、优化器、batch、上下文长度、精度和硬件绑定在一起。交叉熵目标简单，但训练配方高度耦合。一个实验结果必须说明这些条件，才有可复现意义。

6.5.2 目标记账与数据顺序

训练 token 数、unique token 数、epoch、sequence length、effective batch size 和数据顺序都影响学习。数据重复会让某些样本被过度加权，数据顺序会影响训练早期梯度。报告“训练了多少 token”仍不足以解释模型行为。

6.5.3 AdamW 与参数组

AdamW 解耦权重衰减和自适应梯度更新，是大模型训练常用优化器。实践中通常对 bias、norm 参数和 embedding 设置不同 decay 规则。参数组错误会改变正则化，导致难以解释的收敛差异。

6.5.4 Schedule、Warmup 与 Batch Size

Warmup 防止训练初期更新过大，cosine decay 等 schedule 控制后期收敛。Batch size 增大可以提高硬件效率，但也会改变优化噪声和学习率需求。有效 batch 还受到 gradient accumulation 和数据并行规模影响。

6.5.5 梯度裁剪与稳定性

梯度裁剪限制异常更新，混合精度需要 unscale 后再裁剪。NaN、inf、loss spike、激活爆炸和 router collapse 都应被监控。稳定训练不是一个技巧，而是初始化、归一化、精度、优化器和数据共同作用的结果。

6.5.6 Checkpoint 与可复现性

可恢复训练需要保存模型权重、优化器状态、scheduler、随机数状态、数据加载位置、并行配置和 tokenizer 版本。只保存权重无法复现训练轨迹。大规模训练中，checkpoint 频率还必须平衡 I/O 成本和故障恢复成本。

6.5.7 计算最优规划

Scaling laws 帮助在给定计算预算下选择参数量和 token 数。Chinchilla 结果提醒研究者，过大参数而训练 token 不足会浪费计算。规划预训练时应先确定目标损失、数据规模和可用硬件，而不是只追求最大参数。

6.5.8 监控、评测与停止规则

Dashboard 应区分优化健康和能力质量。前者看 loss、梯度、更新、吞吐、显存和通信；后者看代码、数学、多语言、安全和事实性切片。停止训练不应只看训练 loss，还应看验证趋势、能力回归和成本收益。

6.5.9 实现笔记

从小模型开始实现可以暴露标签、mask、dtype 和 checkpoint 错误。扩大规模前，应先验证过拟合小 batch、恢复训练一致性、数据切片 loss 和生成样例。实现纪律比单次大规模运行更能培养可靠判断。

6.6 关键术语、实现要点与练习

关键术语。 Scaling law 描述 loss 随参数、数据和计算变化的规律；Chinchilla-optimal 指给定计算下参数和数据的更优平衡；AdamW 解耦权重衰减；warmup 防止早期大更新；mixed precision 在吞吐和数值稳定之间权衡。

实现要点。 训练日志应分离优化健康和模型质量；checkpoint 应包含权重、优化器、scheduler、随机数状态和数据位置；实验记录应保存 tokenizer、数据 manifest、模型配置、精度、并行策略和验证切片。

练习。

1. 解释为什么两个相同 FLOP 的训练运行可能不可比较。
2. 设计一个训练 dashboard，列出至少八个优化健康指标和八个质量指标。
3. 写出恢复训练所需的 checkpoint 字段。
4. 说明 BF16 相对 FP16 的优势和 FP8 的额外风险。

6.7 结构化检查表

6.7.1 训练 dashboard

类别	指标
优化健康	training loss、validation loss、gradient norm、update norm、learning rate、clipping rate、tokens/s、data-loader wait、communication time、checkpoint time
模型质量	代码、数学、多语言、长上下文、事实性、安全、拒答边界、RAG faithfulness、生成样例、回归测试
系统成本	GPU 利用率、MFU、显存峰值、网络带宽、I/O、失败恢复时间、能耗、美元成本

第七章 分布式训练系统

7.1 内存与并行

训练内存由参数、梯度、优化器状态、激活、临时 buffer 和通信 buffer 组成。数据并行复制模型并对应梯度；ZeRO/FSDP 切分参数、梯度和优化器状态 [70, 101]；tensor parallelism 切分层内矩阵；pipeline parallelism 切分深度；expert parallelism 处理 MoE 专家路由。

每种并行方式都在移动瓶颈。数据并行简单，但每个 worker 仍要保存完整模型状态；FSDP/ZeRO 节省显存，但增加 gather/scatter 和 checkpoint 复杂度；tensor parallelism 降低单卡矩阵尺寸，但引入频繁层内通信；pipeline parallelism 可以容纳更深模型，却产生 bubble 和调度复杂度；expert parallelism 适合 MoE，但 all-to-all 和负载均衡会成为核心问题。

7.2 通信与吞吐

大规模训练不是只买更多 GPU。互联带宽、all-reduce、all-to-all、pipeline bubble、activation checkpointing、kernel fusion 和数据加载都会限制吞吐。报告训练效率时，应同时给出 tokens/s、step time、MFU、GPU 利用率、显存峰值和失败恢复策略。

吞吐还必须和样本效率一起看。某个并行方案可能让 tokens/s 更高，却迫使 batch size 变大、学习率重调或数据顺序改变，最终在相同 token 预算下质量更差。系统优化的正确比较应固定训练目标和质量阈值，报告达到目标 loss 或目标 benchmark 所需的总时间、总能耗、总成本和失败次数。

7.3 实现纪律

CS336 把资源核算、GPU、kernel、Triton、并行和 scaling laws 放在核心位置，是因为语言模型工程的很多结论只有在实现后才显现 [78]。一个研究者应能从 CPU 小模型调试开始，再逐步加入 GPU、混合精度、FlashAttention、checkpointing 和分布式训练。

这种从零实现不是为了重复造轮子，而是为了识别抽象边界。使用成熟框架时，很多关键决定被隐藏在 dataloader、collator、attention kernel、optimizer wrapper 和 distributed launcher 中。只有写过最小版本，研究者才容易发现 loss mask 错误、padding 泄漏、通信等待、显存碎片和 checkpoint 不可恢复等问题。

7.4 深入展开：分布式训练的核心是记账

本章的主线是“accounting”。参数、梯度、优化器状态、激活、临时 buffer 和通信 buffer 都占显存；矩阵乘法、attention、all-reduce、all-to-all、checkpoint I/O 和数据加载都占时间。所谓并行策略，就是把这些字节和时间从一个位置移到另一个位置。说某个方法“省显存”是不够的，必须说明省的是参数、梯度、优化器还是激活，又增加了什么通信和重算。

数据并行最容易理解：每张卡保留模型副本，处理不同 batch，再对应梯度。它简单但显存重复。ZeRO/FSDP 把参数、梯度和优化器状态切分到不同 worker，节省显存但增加 gather、scatter 和 checkpoint 复杂度。Tensor parallelism 切分层内矩阵，适合单层太大无法放入一张卡的情况；pipeline parallelism 切分网络深度，但会产生 pipeline bubble；expert parallelism 服务于 MoE，但 all-to-all 和负载均衡会成为瓶颈。

本章要求训练报告给出 tokens/s、step time、MFU、GPU 利用率、显存峰值、通信时间和失败恢复策略。吞吐不能脱离质量解释：一个方案提高 tokens/s，却强迫使用过大 batch 或改变数据顺序，可能在相同 token 预算下质量更差。正确比较应报告达到某个 loss 或能力阈值所需的总时间、总成本和失败率。

实现纪律还包括故障处理。大规模训练必然遇到节点失败、网络抖动、文件系统慢、数据样本损坏和非确定性重启。checkpoint 频率、rank 对应、日志归并和恢复测试都应在训练开始前设计。一个不能稳定恢复的训练系统，即使单步吞吐很高，也不适合大规模运行。

7.5 章节细节

7.5.1 为什么训练系统是模型的一部分

训练系统决定可训练模型的大小、数据吞吐、故障恢复和成本。相同架构在不同并行策略、精度和数据管线下可能得到不同结果。系统不是外部工程细节，而是模型配方的一部分。

7.5.2 内存记账

显存由参数、梯度、优化器状态、激活、临时 buffer 和通信 buffer 组成。优化器状态常常比参数本身更占空间，激活随 batch 和序列长度增长。没有内存记账，就无法选择并行策略。

7.5.3 数据并行

数据并行让每个 worker 处理不同 batch，再对应梯度。它实现简单，扩展性好，但每个 worker 保存完整模型和优化器状态。规模继续增大时，显存重复会成为瓶颈。

7.5.4 ZeRO 与 FSDP

ZeRO 和 FSDP 切分参数、梯度和优化器状态，显著降低单卡显存。代价是更多 gather、scatter、通信和 checkpoint 复杂度。它们适合模型状态过大但通信网络足够强的场景。

7.5.5 Tensor Parallelism

Tensor parallelism 切分层内矩阵，使单层计算分布到多张卡。它能容纳更宽模型，但引入频繁通信。是否值得使用取决于矩阵大小、互联带宽和 kernel 实现。

7.5.6 Pipeline Parallelism

Pipeline parallelism 按深度切分模型，让不同 stage 处理不同 microbatch。它能容纳更深网络，但会产生 pipeline bubble 和调度复杂度。microbatch 数、stage 平衡和激活传输决定效率。

7.5.7 Expert Parallelism

Expert parallelism 服务于 MoE，把不同专家放在不同设备上。每个 token 经 router 分配到专家，通常需要 all-to-all 通信。负载均衡和专家利用率是核心监控指标。

7.5.8 激活检查点与重算

Activation checkpointing 不保存部分中间激活，在反向传播时重算，从而用计算换显存。它适合显存瓶颈明显的训练，但会降低 tokens/s。报告应说明节省了多少显存、增加了多少计算。

7.5.9 精度与通信

BF16、FP16、FP8 和量化通信改变吞吐与稳定性。低精度能提高速度、降低带宽，但会引入溢出、下溢和缩放问题。通信压缩也必须检查最终质量是否受损。

7.5.10 吞吐记账

吞吐不只是 tokens/s，还包括 step time、MFU、GPU 利用率、数据加载等待、通信时间和失败重启。高吞吐如果伴随质量下降或故障频繁，并不代表系统更好。正确比较应看达到目标质量所需总成本。

7.5.11 运行可靠性

大规模训练必然遇到节点故障、网络抖动、文件系统瓶颈和坏样本。可靠系统需要 checkpoint 恢复测试、日志归并、错误隔离和监报告警。不能恢复的训练系统不适合长时间运行。

7.5.12 实现笔记

实现分布式训练时，应先在单机验证数值，再逐步加入数据并行、FSDP、tensor parallel 和 pipeline。每加一层并行，都要重新检查 loss、梯度、吞吐和 checkpoint。并行不是越多越好，而是为具体瓶颈服务。

7.6 关键术语、实现要点与练习

关键术语。 Data parallelism 复制模型并对应梯度；FSDP/ZeRO 切分参数、梯度和优化器状态；tensor parallelism 切分层内矩阵；pipeline parallelism 切分网络深度；expert parallelism 支持 MoE 专家路由。

实现要点。 所有并行策略都要说明节省哪些字节、增加哪些通信、如何 checkpoint、如何恢复；吞吐报告应包含 tokens/s、step time、MFU、GPU 利用率、显存和失败恢复；系统比较应固定质量目标。

练习。

1. 估算一个 7B 模型在 AdamW 下参数、梯度和优化器状态的显存。
2. 比较 FSDP 与 tensor parallelism 的通信模式。
3. 解释 pipeline bubble 产生的原因，并给出减少方法。

4. 为 MoE 训练列出 all-to-all、负载均衡和专家冷启动风险。

7.7 结构化检查表

7.7.1 并行策略诊断表

策略	节省什么	增加什么
数据并行	提高 batch 吞吐	梯度对应、模型状态重复
FSDP/ZeRO	参数、梯度、优化器显存	gather/scatter、checkpoint 复杂度
Tensor parallel	单层矩阵显存和计算	层内通信、拓扑敏感
Pipeline parallel	单卡容纳深层模型	bubble、调度复杂度
Expert parallel	MoE 专家容量	all-to-all、负载均衡、冷专家
Activation check-pointing	激活显存	重算时间、确定性要求

第八章 推理与服务

8.1 Prefill 与 Decode

推理服务不是“关闭 dropout 后跑模型”。Prefill 处理 prompt，代价随输入长度增长；decode 逐 token 生成，代价随输出长度和 batch 调度变化。真实服务必须处理流式输出、取消、超时、缓存、租户隔离和安全过滤。

首 token 延迟主要受 prefill、排队和调度影响；后续 token 速度主要受 decode、KV cache 和 batch 形状影响。长文档总结、短聊天、代码补全、RAG 和 agent 工具循环对服务系统提出的压力完全不同。如果所有请求进入同一个 FIFO 队列，长上下文请求可能阻塞短请求；如果过度偏向短请求，长任务可能永远得不到足够资源。

8.2 KV Cache、批处理与调度

KV cache 是长上下文推理的主要内存压力。PagedAttention 等方法把 cache 管理变成类似操作系统内存分页的问题 [43]。服务系统需要在吞吐、首 token 延迟、尾延迟和公平性之间平衡。

调度器需要 admission control。它必须在请求进入时估计 prompt length、max new tokens、并发数和 KV cache 上限；还要在生成过程中处理提前停止、用户取消、工具等待和安全检查。结构化输出和工具调用还会引入解析器、schema validator、沙盒执行和重试逻辑，这些都应计入延迟预算。

8.3 下一代推理系统

随着 MoE、长上下文和 disaggregated serving 发展，推理服务开始分离 prefill/decode、attention/FFN 或专家路由。Frontier simulator 这类工作说明，下一代服务系统必须能模拟专家并行、跨集群路由和异构资源 [23]。

生成式多模态系统还会扩大服务问题。图像编码、视频采样、ASR、语音合成、扩散步

数、flow matching 调度和安全滤镜都可能与文本模型竞争资源。一个 Omni 系统的“延迟”不是单一模型 decode 时间，而是编码器、connector、LLM prefill、LLM decode、语音/图像/视频生成器和后处理共同构成的端到端时间。

8.4 深入展开：推理服务是另一套系统工程

本章把推理和服务从训练中分离出来。训练时目标是高吞吐地处理大量 token；服务时目标是满足用户请求的延迟、成本、可靠性和安全边界。Prefill 处理 prompt，一次性计算所有输入 token；decode 每次生成一个新 token，需要反复调度。短聊天、长文档总结、代码生成、RAG、agent 工具循环和语音对话的服务形态完全不同。

KV cache 是服务成本核心。每个活动请求都占用与层数、head 数、head 维度、上下文长度和 batch 大小相关的 cache。PagedAttention 把 cache 管理变成类似操作系统分页的问题，减少碎片并提高 batch 利用率 [43]。但调度器仍需决定哪些请求可以进入、如何混合长短请求、如何处理取消和超时、如何在多租户场景下保证隔离。

采样和结构化输出也属于服务逻辑。温度、top- p 、repetition penalty、logit bias、stop sequence 和随机种子应有确定顺序。JSON、SQL、函数调用和工具参数需要增量验证；一旦输出不合法，系统要决定修复、重试、拒绝还是交给人工。安全过滤可能发生在 prompt 前、流式生成中和最终输出后，每一步都会增加延迟和失败模式。

下一代推理系统还会遇到 MoE 和 disaggregated inference。Prefill 和 decode 的硬件需求不同，专家路由和 FFN 可能适合不同资源，长上下文请求可能需要特殊调度。本章强调，服务研究不应把模型当作固定黑盒，而应研究架构如何改变调度器、缓存、通信和成本模型。

8.5 章节细节

8.5.1 服务不是 Evaluation Mode

推理服务不是把模型切到 eval 后运行。服务系统要处理请求排队、流式输出、取消、超时、缓存、配额、日志、安全过滤和多租户隔离。用户体验由端到端系统决定，而不是单次 forward 决定。

8.5.2 Prefill 与 Decode

Prefill 处理输入 prompt，decode 逐 token 生成输出。Prefill 更像大矩阵并行计算，decode 更受调度和 KV cache 约束。长输入短输出、短输入长输出和多轮工具调用的瓶颈

不同。

8.5.3 KV Cache

KV cache 保存历史 key/value，避免每步重复计算前缀。它随层数、头数、head 维、上下文长度和并发请求增长。长上下文服务的主要显存压力往往来自 cache，而不是模型权重。

8.5.4 Batching 与调度

连续 batching 让不同请求在生成过程中动态合并，提高 GPU 利用率。调度器必须兼顾长短请求、公平性、超时和吞吐。一个追求最大吞吐的策略可能让短请求延迟不可接受。

8.5.5 内存管理与准入控制

PagedAttention 等方法把 KV cache 管理成分页式 block，减少碎片。准入控制决定新请求能否进入系统，防止 cache 爆掉或延迟失控。服务质量依赖调度、缓存和配额共同控制。

8.5.6 量化与压缩

权重量化、KV cache 量化和剪枝可以降低成本，但也可能影响事实性、数学、代码和安全边界。量化评测应覆盖目标部署任务，而不是只看平均 benchmark。不同硬件对量化格式支持也不同。

8.5.7 投机与并行解码

Speculative decoding 用小模型提出候选，再由大模型验证，以减少大模型调用次数。它提高速度的前提是草稿模型足够匹配目标模型。并行解码和多 token prediction 也需要质量、延迟和复杂度权衡。

8.5.8 注意力 Kernel 与长上下文

FlashAttention、滑动窗口、分块注意力和压缩上下文都会改变成本曲线。长上下文不是只改变最大长度，还改变 prefill 时间、cache 体积和调度策略。服务评测应使用真实长度分布。

8.5.9 流式 API、取消与安全过滤

流式输出提升感知速度，但也要求中途安全过滤和可取消执行。用户取消后，系统必须释放 cache 和工具资源。安全过滤可能发生在输入、生成中和输出后，每个位置都有不同延迟和漏检风险。

8.5.10 负载测试与成本记账

服务报告应给出 TTFT、TPOT、吞吐、p50/p95/p99 延迟、并发、上下文长度分布和成本。平均延迟不足以解释用户体验。成本也应分解为权重常驻、prefill、decode、cache、工具和后处理。

8.5.11 实现笔记

实现服务时，应先用固定 prompt 和固定 seed 建立基线，再加入流式、batching、量化、RAG 和工具。每加一个功能都要重新测延迟、显存和失败模式。服务系统的正确性包括输出正确，也包括资源正确释放。

8.6 关键术语、实现要点与练习

关键术语。 Prefill 处理 prompt；decode 逐 token 生成；KV cache 保存历史 key/value；paged attention 管理 KV block；admission control 决定请求能否进入系统；tail latency 反映最慢用户体验。

实现要点。 服务应分别测量排队、prefill、decode、工具、过滤和后处理延迟；调度器应处理长短请求混合、取消、超时和多租户隔离；结构化输出、安全过滤和工具调用都应进入成本模型。

练习。

1. 估算一个长文档请求的 KV cache 显存。
2. 设计一个混合流量调度策略：短聊天、长总结、代码生成和 RAG。
3. 说明为什么 p95/p99 latency 比平均 latency 更重要。
4. 为一个 tool-use 服务写出失败恢复策略：参数非法、工具超时、返回冲突和权限不足。

8.7 结构化检查表

8.7.1 推理服务成本分解

阶段	需要测量
排队	admission control、优先级、租户隔离、请求混合
Prefill	prompt tokens、视觉 token、检索证据、batch shape
Decode	输出长度、KV cache、采样、stop condition、streaming
工具	schema validation、网络延迟、超时、重试、沙盒
安全	prompt filter、streaming filter、final classifier、人工升级
后处理	citation check、JSON repair、logging、monitoring

第三部分

适配

第九章 监督指令微调

9.1 接口转换

SFT 把基础模型变成遵循指令的模型。一个样本包含系统消息、用户消息、上下文、工具观察和目标回答。训练时通常只对 assistant token 计算损失，prompt token 作为条件而非标签。

chat template 是数据格式的一部分。相同语义样本如果被写成 system/user/assistant 消息、普通 prompt-completion、工具调用 JSON 或厂商私有格式，会训练出不同边界行为。训练和推理模板不一致，是让一个本来可用的模型表现异常的常见原因。模板应明确角色、轮次结束、工具观察、图像占位符和停止条件。

9.2 合成数据

Self-Instruct、WizardLM、Orca 和 STaR 显示了合成指令、复杂任务演化、教师解释蒸馏和自举推理数据的价值 [86, 89, 56, 97]。合成数据管线必须记录生成器、prompt、采样参数、过滤规则、拒绝原因和评测重叠。

合成数据的风险随规模放大。教师模型会编造事实、写出不安全代码、误拒合法请求或把自身风格强加给学生模型。可靠管线通常需要四道门：生成 prompt 规定任务族、难度、来源和格式；自动过滤去掉重复、过短、格式错误或越界样本；任务验证器执行代码、SQL、JSON 或数学检查；人工抽样检查估计残余错误。目标不是让数据看起来多样，而是补上当前模型缺少且可验证的覆盖。

合成指令数据可以写成“提议分布加过滤器”。生成器从种子样本和生成策略中采样候选 $e \sim q_\omega(e | s, \pi)$ ，再由验证器 $v_1, \dots, v_K$ 接受或拒绝：

$$\mathcal{D}_{\text{accept}} = \{e : v_k(e) = 1 \text{ for all required gates } k\}.$$

因此，合成数据质量不由流畅度决定，而由验证门决定。代码样本需要执行测试，数学样本需要可检查答案，安全样本需要相邻合法/非法案例，RAG 样本需要证据支持。

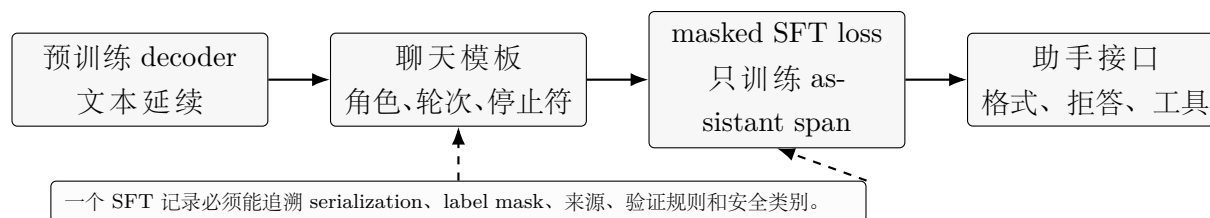


图 9.1: SFT 改变的是接口分布，而不是自回归语言建模的基本分解。

9.3 局限

SFT 学会的是示范分布。它可能提升格式和礼貌，却不一定提升真实偏好、事实性或安全性。评估 SFT 不能只看指令榜单，也要看基础能力回归、拒答边界、长上下文、多语言和代码能力。

SFT 还会产生能力遗忘。一个模型在聊天格式上变好，可能同时降低翻译、代码、事实召回、数学或低资源语言能力。训练集越窄、越合成、越单一语言，回归风险越高。严肃报告应包含 base model 与 tuned model 的对照，并在未优化任务上保留回归测试。

9.4 深入展开：SFT 是接口训练，不是完整对齐

本章把监督指令微调定义为接口转换。预训练模型已经能延续文本，但它不知道什么是 system instruction、什么是用户请求、什么是可接受回答、什么是工具返回。SFT 通过示范把这些格式变成条件生成问题。训练时通常只对 assistant token 计算 loss，让用户问题、系统消息和检索证据作为条件，而不是让模型学习“预测用户”。

Chat template 是本章的关键。模板应明确角色、轮次、结束符、工具调用、工具观察、多模态占位符和安全上下文。训练模板与推理模板不一致，会让一个本来正常的模型出现角色混淆、提前结束、工具调用格式错误或拒答边界失效。因此要求把模板和 tokenizer、special tokens、模型权重一起版本化。

合成数据扩展覆盖，但也复制教师错误。Self-Instruct 通过少量种子指令生成大量任务；WizardLM 类方法把任务演化得更复杂；Orca 类蒸馏让学生学习更丰富的教师解释；STaR 用正确答案筛选和迭代生成推理轨迹。它们共同说明，现代 SFT 数据管线是生成、过滤、验证和检查的系统，而不是一次性调用一个强模型。

本章还提醒，模仿不能替代偏好和真实性。SFT 可以让回答更像人类示范，但示范本身可能过时、错误、风格单一或安全边界含糊。模型可能学会“看起来有帮助”的语气，却没有更强事实性；也可能学会教师模型的过度拒答。SFT 后必须做独立评测，覆盖事实、代码、数学、多语言、长上下文、安全和未优化任务。

9.5 章节细节

9.5.1 接口转换

SFT 把预训练模型转化为遵循指令的接口模型。它通过示范学习角色、格式、拒答、工具调用和回答风格。训练时通常只在 assistant token 上计算 loss，让其他消息作为条件。

9.5.2 指令数据作为契约

一条指令样本不只是 prompt 和 answer，还包含系统策略、角色边界、上下文、目标格式和安全期望。数据格式定义了模型未来如何解释用户请求。模板和 special tokens 必须版本化。

9.5.3 合成指令数据

合成数据能扩大任务覆盖，但会复制教师错误和风格偏置。Self-Instruct、WizardLM、Orca 和 STaR 展示了生成、演化、蒸馏和验证的不同策略。合成数据必须过滤、去重、检查和独立评测。

9.5.4 训练细节

SFT 训练要处理 loss mask、packing、学习率、epoch、长度分布和混合数据。过多 epoch 可能导致风格过拟合和能力遗忘。应比较 base model 与 tuned model 在未优化任务上的回归。

9.5.5 拒答与安全数据

安全数据定义模型何时拒答、如何提供安全替代、如何处理敏感但合法的请求。过少会造成越界，过多会造成误拒。拒答质量应按政策类别和用户意图切片评测。

9.5.6 评测

SFT 评测应覆盖格式遵循、事实性、代码、数学、多语言、安全、长上下文和回归任务。单纯看聊天偏好容易奖励礼貌和冗长。独立测试集和人工错误分类仍然必要。

9.5.7 模仿的局限

SFT 学的是示范分布，不保证真实性、最优性或安全性。示范者会犯错，教师模型也会产生幻觉。后续偏好学习、验证器、RAG 和工具约束常用于弥补这些局限。

9.6 关键术语、实现要点与练习

关键术语。 Demonstration data 是示范回答；assistant-only loss 只训练助手输出；self-instruction 用模型生成新任务；distillation 从教师模型学习行为；capability regression 指微调后基础能力下降。

实现要点。 SFT 数据应记录模板、生成器、过滤器、验证器和人工检查；合成数据必须检测事实错误和安全边界；评测要包含未优化任务回归；模板版本必须和模型权重一起保存。

练习。

- 1. 构造一个包含工具观察的 SFT 样本，并标出 loss mask。
- 2. 设计一个合成数据过滤管线，包含格式、重复、事实和安全检查。
- 3. 找二十个 SFT 样本，判断它们是否真正增加任务覆盖。
- 4. 设计 SFT 前后回归测试，覆盖代码、数学、多语言和拒答。

9.7 结构化检查表

9.7.1 SFT 数据类型与风险

数据类型	价值	风险
人工示范	高质量、符合目标风格	贵、覆盖有限、标注者偏差
合成指令	覆盖快、格式多	教师幻觉、风格单一、污染
工具轨迹	学习函数调用与恢复	虚假工具结果、权限混淆
安全样本	学习拒答和重定向	过拒、边界模糊
领域示范	学习术语和流程	浅层风格模仿，无证据支持

第十章 参数高效适配

10.1 Adapter、Prompt 与 LoRA

PEFT 的目标是减少训练参数、显存和 checkpoint 体积。Adapter 在层内加入小模块；prefix/prompt tuning 学习连续提示；LoRA 把权重更新写成低秩矩阵乘积 $\Delta W = BA$ [34]. 它适合多任务共享基础模型，但不免除数据和评测责任。

LoRA 的 rank、alpha、dropout 和 target modules 都是建模选择。只调 query/value projection 成本较低，但对代码、多语言或结构化生成可能不够；同时调 attention 和 MLP 更强，但更容易过拟合或破坏基础能力。adapter 合并到基础权重后，也要重新评估量化、服务延迟和安全过滤，因为合并改变了实际部署 artifact。

Rank r	LoRA 参数 $2rd$	Full matrix d^2 ($d = 4096$)	比例
4	32,768	16,777,216	0.20%
8	65,536	16,777,216	0.39%
16	131,072	16,777,216	0.78%
64	524,288	16,777,216	3.12%

10.2 QLoRA 与量化训练

QLoRA 把基础模型量化为低精度并训练 LoRA adapter，从而显著降低显存需求 [19]. 量化带来效率，也带来数值误差、目标模块选择和部署合并问题。报告 PEFT 结果时，应同时给出任务指标、回归指标、显存、训练时间、checkpoint 大小和服务延迟。

PEFT 容易让实验看起来便宜，但数据和评测仍然决定可信度。一个 20MB adapter 可以携带强烈风格、过时知识或安全回归。多 adapter 系统还需要版本管理、冲突处理和权限控制：不同用户、领域或任务加载不同 adapter 时，系统必须知道它们基于哪个基础模型、哪个 tokenizer、哪个 chat template 和哪个安全策略训练。

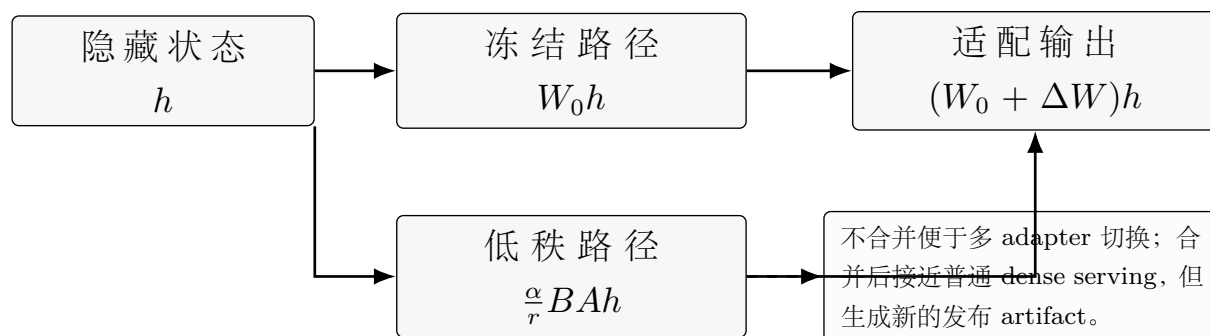


图 10.1: LoRA 在冻结矩阵旁增加低秩残差路径。训练省参数不等于部署无代价，adapter 管理、合并策略和回归测试仍然是发布对象。

10.3 深入展开：参数高效不等于风险高效

本章解释 PEFT 的动机：全参数微调需要大量显存、训练时间和 checkpoint 存储，而很多适配任务只需要改变少量行为。Adapter 在层中插入小模块，prefix/prompt tuning 学习连续提示，LoRA 用低秩矩阵表示权重更新。它们共同的目标是减少可训练参数和部署 artifact 大小。

LoRA 的数学形式 $\Delta W = BA$ 很简单，但工程选择很多。Rank 决定更新容量，alpha 决定缩放，dropout 影响正则，target modules 决定修改 attention、MLP 还是更多层。只调 query/value projection 省钱，但可能不足以适配复杂代码和多语言；调更多模块更强，但回归风险更高。不同任务的最佳 rank 和 target set 不能直接迁移。

QLoRA 把基础权重量化到低精度，再训练 LoRA adapter，使单机显存也能适配较大模型。但量化会引入数值误差，训练时的量化配置、推理时的加载方式、adapter 合并与否都会影响结果。本章要求报告显存、训练时间、checkpoint 大小、延迟和回归指标，而不是只报告任务分数。

PEFT 的治理问题常被低估。Adapter 很小，所以容易被大量创建、下载、组合和热切换。一个 adapter 可以改变安全边界、输出风格或领域知识；多个 adapter 叠加可能相互冲突。生产系统应记录每个 adapter 的基础模型、tokenizer、模板、数据来源、训练目标、评测结果和允许使用的场景。

10.4 章节细节

10.4.1 到底在让什么高效

PEFT 让可训练参数、显存、训练时间和 checkpoint 体积变小，但不自动降低数据风险和评测成本。它适合多任务共享基础模型，也适合资源有限场景。效率指标必须和质量、

延迟、治理一起报告。

10.4.2 Adapter 与 Prompt 家族

Adapter 在模型层中插入小模块，prefix/prompt tuning 学习连续条件向量。它们减少可训练参数，但也改变推理路径或上下文占用。选择方法时应看任务复杂度、部署形态和是否允许修改模型结构。

10.4.3 LoRA

LoRA 把权重更新写成低秩矩阵乘积，冻结基础权重，只训练小矩阵。Rank、alpha、dropout 和 target modules 决定容量与风险。LoRA 不是只调一个 rank，而是一组训练和部署选择。

10.4.4 QLoRA 与量化训练

QLoRA 通过低精度基础权重和 LoRA adapter 降低显存。NF4、double quantization 和分页优化器等机制服务于可训练性。量化配置需要和推理加载方式一致，否则评测结果可能不可复现。

10.4.5 选择 Rank、目标模块与超参数

低 rank 省资源但可能欠拟合，高 rank 更强但增加回归和存储。只调 attention 投影、省钱但能力有限；调 MLP 或更多层可能适合复杂任务。选择应基于验证集和切片评测，而不是经验数字。

10.4.6 系统成本与部署

多 adapter 系统需要加载、合并、热切换、权限控制和版本管理。Adapter 小不等于风险小，它可以改变安全边界和领域行为。部署报告应记录基础模型、tokenizer、模板、数据、评测和适用范围。

10.4.7 适配模型评测

PEFT 评测不能只看目标任务提升，还要看基础能力回归、延迟变化、安全变化和跨任务干扰。Adapter 合并前后也应分别评测。对于多租户服务，还要验证 adapter 隔离和权限。

10.5 关键术语、实现要点与练习

关键术语。 Adapter 是插入层内的小模块；prefix tuning 学习连续前缀；LoRA 用低秩矩阵表示权重更新；QLoRA 在量化基础模型上训练 LoRA；target modules 指 LoRA 注入的位置。

实现要点。 PEFT 报告应包含 rank、alpha、dropout、target modules、量化配置、显存、训练时间、服务延迟和回归指标；多 adapter 系统应记录基础模型和安全策略；adapter 小不代表风险小。

练习。

- 1. 对一个线性层写出 LoRA 更新的矩阵形状。
- 2. 比较只调 attention projection 和同时调 MLP 的风险。
- 3. 设计一个 QLoRA 实验报告模板。
- 4. 说明多个 adapter 叠加时可能出现的冲突。

10.6 结构化检查表

10.6.1 PEFT 报告模板

字段	内容
基础模型	checkpoint、tokenizer、chat template、量化方式
Adapter 配置	LoRA rank、alpha、dropout、target modules、是否合并
训练数据	来源、规模、语言、领域、合成比例、污染检查
训练成本	显存、时间、GPU、batch、学习率、精度
质量评测	目标任务、未优化任务、安全、事实性、代码/数学/多语言回归
部署影响	checkpoint 大小、加载策略、延迟、吞吐、adapter 冲突

第十一章 领域与语言适配

11.1 适配是一种诊断

模型在新领域失败，可能因为 tokenizer 对语言切分差，预训练缺少事实，风格不匹配，输出结构不稳定，或答案必须基于私有知识。不同诊断对应不同方案：继续预训练、SFT、LoRA、RAG、工具调用或拒答策略。

因此，领域适配不应从“先微调”开始，而应从失败分类开始。如果失败是术语不熟，继续预训练或领域 SFT 可能有用；如果失败是知识更新，RAG 更合适；如果失败是必须执行数据库查询或计算，工具调用更合适；如果失败是高风险边界，安全策略和人类审核比更多训练更重要。错误诊断会把问题写入权重，反而更难撤回。

11.2 继续预训练与检索

继续预训练适合稳定公开知识和领域语言模式；RAG 适合变化快、私有或需要可追溯证据的知识 [29, 45]。把私有知识写入权重可能带来隐私和更新成本；只做检索又可能受限于召回、chunking 和权限控制。

领域系统常常需要组合方案。医疗问答可以用检索提供最新指南，用 SFT 教模型如何引用证据和表达不确定性，用工具执行药物相互作用检查，用安全策略限制诊断范围。NL2SQL 可以用 schema serialization、示例检索、SFT 和执行验证共同提高可靠性。关键是把知识、行为和权限分层，而不是把所有问题都交给一次微调。

11.3 领域评测

领域评测应按实体、时间、语言、来源、任务类型和风险等级切片。医疗、法律、金融等高风险场景还需要专家审查、拒答质量、证据可追溯和上线监控。

随机切分通常不够。代码任务应按仓库切分，客服任务应按客户或时间切分，NL2SQL 应按数据库或 schema 切分，法律和医疗任务应按时间和管辖范围切分。否则模型可能只

是记住模板、实体或 schema。领域适配成功的标准也不是 benchmark 提升几分，而是模型质量、系统成本、安全边界和治理证据同时改善。

领域适配也应有发布级效用标准。令 ΔQ 表示任务质量提升， ΔC 表示成本增加， ΔR 表示风险增加， ΔG 表示治理负担增加，则一个简化发布标准可以写为

$$U_{\text{adapt}} = \Delta Q - \lambda_C \Delta C - \lambda_R \Delta R - \lambda_G \Delta G > 0.$$

这个公式不是万能效用函数，而是提醒团队：领域 benchmark 上升但系统更慢、更难撤回、更容易泄露隐私或更难检查时，不能直接发布。

11.4 深入展开：领域适配先诊断再训练

本章强调，领域适配的第一步是诊断失败类型。模型可能因为 tokenizer 对目标语言效率低而失败，可能因为预训练语料缺少领域事实而失败，可能因为输出格式不稳定而失败，可能因为任务需要私有知识而失败，也可能因为安全策略边界不清而失败。每种失败对应不同方案，不能把微调当作默认答案。

继续预训练适合稳定、公开、规模足够且许可证清晰的领域文本，例如科学文献、专利、代码规范或目标语言网页。它能改变模型内部语言分布，但也把知识写入权重，后续撤回困难。RAG 适合变化快、私有、需要权限控制和引用检查的知识，例如企业政策、客户记录、当前价格、医疗指南和法律文件。很多系统需要两者结合：用继续预训练学习语言风格，用检索提供最新证据。

NL2SQL 是本章中的典型例子。模型不仅要生成 SQL，还要理解 schema、列名、实体、聚合、过滤和 join。评测不能只看字符串完全匹配，因为等价 SQL 可以有不同表面形式；执行准确率更有意义，但也会被小数据库偶然掩盖错误。可靠评测应包括 parse validity、execution accuracy、schema linking、沙盒安全和失败样例审查。

高风险领域尤其需要治理。医学模型会说医学词汇，不代表可以做临床决策；法律模型会引用法条，不代表能替代律师；金融模型会解释产品，不代表可以给投资建议。领域适配的发布证据应包含专家评审、拒答边界、引用忠实度、权限控制、日志、监控和回滚，而不仅是排行榜。

11.5 章节细节

11.5.1 适配是一种诊断

领域失败可能来自知识缺失、语言切分差、格式不稳、私有信息不可见、评测分布偏移或安全限制。不同原因对应继续预训练、SFT、LoRA、RAG、工具或拒答策略。先训练

再诊断通常会浪费成本。

11.5.2 语言适配

语言适配要考虑 tokenizer fertility、语料质量、方言、文字系统和跨语言迁移。低资源语言常被过度切分，导致同样语义占用更多 token。评测应覆盖自然文本、任务格式和文化语境。

11.5.3 继续预训练

继续预训练适合稳定、规模足够、许可证清晰的领域语料。它能让模型吸收风格和知识，但也可能遗忘通用能力，并把可变知识写入权重。训练前应决定哪些知识需要权重化，哪些应留给检索。

11.5.4 监督领域微调

领域 SFT 让模型学习任务格式和专家示范。它适合问答、报告生成、客服、代码规范和结构化输出。风险是示范数据过窄导致模型学会模板而非能力。

11.5.5 结构化任务：NL2SQL

NL2SQL 需要理解 schema、列名、实体、过滤、聚合和 join。字符串匹配不足以评测等价 SQL，执行准确率也可能被数据偶然性误导。安全沙盒和权限控制是部署前提。

11.5.6 检索还是更新权重

RAG 适合频繁变化、私有、需引用和权限控制的知识；权重更新适合稳定语言模式和通用领域表达。很多系统需要混合方案。决策依据应是知识变化速度、可检查性、成本和安全边界。

11.5.7 领域安全与治理

医疗、法律、金融、教育和政务等场景不能只看任务分数。模型需要拒答边界、引用、专家审查、日志、监控和回滚。领域术语流畅并不等于具备专业责任能力。

11.5.8 分布偏移下的评测

随机切分常常高估领域能力，因为同一客户、仓库、schema 或时间段的信息泄漏到测试集。应按时间、实体、文档源或数据库切分。评测还应包含真实失败样例和人工审查。

11.6 关键术语、实现要点与练习

关键术语。 Continual pretraining 继续语言模型目标；domain adaptation 改善领域语言 and 任务；RAG 把知识保留在可更新证据库；NL2SQL 把自然语言映射为 SQL；domain shift 指部署分布和训练分布不同。

实现要点。 适配前先诊断失败类型；私有或快速变化知识优先检索；领域评测应按实体、时间、语言、来源和风险切片；高风险领域需要专家审查和上线监控。

练习。

- 1. 给一个领域失败案例判断应使用继续预训练、SFT、RAG、工具还是拒答策略。
- 2. 设计一个 NL2SQL benchmark split，避免 schema 泄漏。
- 3. 为医疗问答系统列出证据、拒答和专家审查要求。
- 4. 比较中文 tokenizer 扩展和继续预训练的成本与风险。

11.7 结构化检查表

11.7.1 领域适配决策表

失败原因	首选方案	注意事项
领域术语少见	继续预训练或 SFT	检查许可证和回归
知识频繁变化	RAG	检索召回、权限、引用
私有事实	RAG 或工具	不宜写入权重
格式不稳定	SFT 或 constrained decoding	模板和解析器一致
需要计算/执行	工具调用	沙盒、权限、错误恢复
高风险边界	安全策略和人工审查	不靠微调替代治理

第四部分

对齐、应用与评测

第十二章 检索、工具与 Agent

12.1 RAG 作为控制系统

RAG 把检索器、reranker、上下文构造器和生成器连接起来。核心指标不是“回答看起来对”，而是检索 recall、上下文 precision、答案正确性、引用忠实度、拒答行为和延迟成本 [38, 45, 24]。

12.2 上下文工程与记忆

RAG 只是更大问题的一种形式：上下文工程。上下文工程把送入模型的全部信息载荷视为可设计、可压缩、可路由、可检查的对象，包括 system instruction、developer policy、用户请求、检索文档、对话历史、工具输出、临时 scratchpad、长期记忆和结构化状态。最新综述把这件事和早期的“prompt engineering”区分开来：真正的问题不是写一句更漂亮的提示词，而是设计一个有权限、来源、时效和失效处理的上下文管线 [54]。

这个区分很重要，因为模型足够强并不代表上下文契约正确。长提示可能包含证据，却把证据放在模型不容易利用的位置；记忆系统可能保留用户偏好，也可能保留过期或敏感信息；工具输出可能在生成时已经变旧；压缩器可能减少延迟，却删除了本应导致拒答的不确定性。因此，生产系统不应把记忆写成一个无差别文本缓冲区，而应区分用户偏好、任务状态、会话摘要、文档向量、工具观察和安全决策，并为每类记忆规定来源、更新规则、过期时间、用户可见性、删除语义和权限检查。

长上下文模型不能取代上下文工程。 ∞ Bench、RULER 和 LongBench v2 等评测显示，名义上下文长度和有效上下文利用能力并不相同，尤其在聚合、多跳推理、矛盾处理和真实长文档理解任务上差异明显 [99, 33, 8]。一个能接受 256K 或 1M token 的模型，仍然可能无法跨章节绑定证据、处理干扰信息或找到非字面匹配的事实。很多系统中，更短但经过检索、chunking、排序和状态管理的上下文，比原样塞入长文档更可靠。

12.3 工具使用

Toolformer、ReAct 和 WebGPT 等工作展示了模型调用工具、观察结果并继续推理的模式 [72, 94, 57]. 工具系统必须管理权限、参数验证、超时、重试、日志、状态和安全边界。

12.4 Agent workflow

Agent 不是一个魔法能力, 而是循环: 选择动作、执行工具、观察结果、更新状态。Kimi K2 等模型报告把 agentic coding、长上下文和工具使用作为核心能力 [40]. 评测 agent 时, 应报告成功率、工具调用次数、成本、延迟、失败模式和权限违规。

12.5 深入展开: RAG、上下文和工具是一个控制系统

本章把 RAG 定义为控制系统, 而不是“把文档塞进 prompt”。一个 RAG 系统要决定语料范围、chunk 大小、metadata、embedding 模型、稀疏/稠密检索、hybrid score、reranker、上下文排序、引用格式、拒答规则和 prompt injection 防护。每个组件都可能失败, 必须分别评测。

检索评测先于生成评测。如果支持答案的证据没有进入候选集合, 生成器再强也只能猜。Recall@k、MRR、nDCG 衡量候选召回; context precision 衡量塞进 prompt 的内容有多少真正相关; answer faithfulness 衡量回答是否被证据支持; citation faithfulness 衡量引用是否真的支撑对应句子。本章特别强调 oracle context baseline: 给模型金标准证据后仍答错, 说明生成器有问题; 检索证据下答错而 oracle 下答对, 说明检索或上下文构造有问题。

上下文工程把 RAG 扩展到完整信息载荷。系统指令、开发者策略、用户请求、检索文档、工具输出、会话历史、长期记忆和结构化状态都可能进入模型上下文。它们不能混成一个无标签文本块。每类上下文都需要来源、权限、时效、转换记录、删除规则和检查路径。长期记忆尤其要 typed: 用户偏好、任务状态、安全决策和文档 embedding 不应共享同一语义。

工具和 agent 把语言模型变成行动系统。Toolformer、ReAct、WebGPT 和现代 coding agent 都显示, 模型可以选择工具、观察结果、更新状态并继续推理。但运行时必须在模型外部验证权限、schema、沙盒和超时。Agent 评测不应奖励“看起来有计划”的 transcript, 而应看任务完成、工具调用数、成本、延迟、回滚、权限违规和最终 artifact 质量。

12.6 章节细节

12.6.1 RAG 作为控制系统

RAG 包含索引、检索、重排、上下文构造、生成、引用和拒答。它不是把文档拼进 prompt，而是一个带有多个可测环节的控制系统。每个环节都需要独立指标和日志。

12.6.2 索引与检索

索引设计包括 chunk、metadata、embedding、稀疏检索、稠密检索和混合检索。Chunk 太小会丢上下文，太大又降低精度和吞吐。检索评测应先看证据是否被召回。

12.6.3 重排与上下文构造

Reranker 改善候选排序，上下文构造决定最终给模型看的证据顺序和格式。系统应去重、合并、标注来源、控制长度并处理冲突证据。上下文质量往往比生成模型规模更影响 RAG 答案。

12.6.4 带证据生成

生成器应基于检索证据回答，不能把模型内部记忆和证据混在一起。引用应支撑具体句子，而不是挂在段落末尾装饰。无法找到证据时，系统应允许拒答或提出澄清。

12.6.5 RAG 评测

RAG 评测包括 retrieval recall、context precision、answer correctness、faithfulness 和 citation faithfulness。Oracle context baseline 能区分检索失败和生成失败。只看最终答案会隐藏系统瓶颈。

12.6.6 上下文工程与记忆

上下文包含系统指令、用户请求、检索证据、工具结果、会话历史和长期记忆。每类信息都应有来源、权限、时效和删除规则。长期记忆不应把偏好、事实、任务状态和安全决策混为一类。

12.6.7 Prompt Injection 与信任边界

外部文档可能包含恶意指令，试图覆盖系统策略或泄露数据。RAG 系统必须把检索内容标为不可信证据，而不是指令。工具权限、引用和数据访问都需要在模型外部强制执行。

12.6.8 工具使用

工具调用把模型输出变成可执行动作。Schema、参数验证、超时、沙盒、权限和错误通道必须由运行时控制。模型可以建议动作，但不能绕过系统授权。

12.6.9 Agent 与 workflow

Agent 是循环：观察状态、选择动作、执行工具、读取结果、更新计划。可靠 agent 评测看任务完成、成本、步骤数、错误恢复和最终 artifact，而不是看 transcript 是否显得聪明。workflow 越长，日志和回滚越重要。

12.6.10 系统成本

RAG 和 agent 会增加检索、重排、工具、长上下文和多轮生成成本。延迟也变成多个组件的总和。成本报告应按组件拆分，才能知道优化点在哪里。

12.7 关键术语、实现要点与练习

关键术语。 Chunk 是检索单位；dense retrieval 用向量相似度；hybrid retrieval 结合稀疏和稠密信号；reranker 对候选重排；context engineering 设计完整上下文载荷；prompt injection 是不可信文本试图越权。

实现要点。 RAG 应分别评测 retrieval recall、context precision、answer correctness、citation faithfulness 和 abstention；工具调用必须由运行时验证 schema 和权限；agent 评测应看任务完成和外部状态，而不是 transcript 是否好看。

练习。

1. 构建 30 个问题的 RAG 小评测，先报告 recall@5。
2. 比较短 chunk、长 chunk 和重叠 chunk 的成本与召回。
3. 设计一个 typed memory schema，包含来源、过期、可见性和删除规则。
4. 构造五个 prompt injection 文档并测试系统是否越权。

12.8 结构化检查表

12.8.1 RAG 设计选择

组件	选择	失败模式
Chunking	长度、重叠、metadata、边界	证据被切断或埋没
Embedding	模型、归一化、语言覆盖	领域词或低资源语言召回差
Retriever	稀疏、稠密、hybrid	漏掉精确 id 或语义改写
Reranker	cross-encoder、LLM judge、规则	延迟高、偏差、过拟合
Context builder	排序、去重、压缩、权限	证据失真或越权
Generator	prompt、引用、拒答	幻觉、citation laundering
Monitor	freshness、drift、logs	索引过期、权限更新慢

第十三章 偏好学习与对齐

13.1 偏好数据

偏好学习把单一目标回答改成候选回答比较。一个 prompt 下的 chosen/rejected 对只表达某个标注规范、标注人群和采样分布下的偏好，不是普遍真理。Reward model 学到的是这种局部偏好函数。

13.2 RLHF、PPO 与 DPO

RLHF 先训练 reward model，再用带 KL 约束的策略优化改善回答 [103, 79, 63]。PPO 是常见算法；DPO 则把偏好优化转化为更直接的监督式目标 [73, 69]。二者都可能出现过优化、奖励黑客、长度偏差和分布漂移。

13.3 DPO 之后的偏好目标

DPO 让偏好优化更像监督学习，但它并没有结束后训练目标的设计空间。后续方法继续追问：是否必须有成对比较？是否必须有冻结 reference model？是否必须显式训练 reward head？是否必须进行在线 rollout？答案取决于数据可得性、系统成本和可接受的失败风险。

KTO 把对齐写成前景理论式优化，使用 desirable/undesirable 样本而不是严格的 chosen/rejected 成对比较 [21]。这适合反馈天然以好/坏、可接受/不可接受形式收集的场景，但也要求仔细检查正负样本是否在任务、语言、风险等级和难度上平衡。ORPO 把 chosen response 的监督学习与抑制 rejected response 的 odds-ratio 惩罚放在同一个目标里，省去单独 reference model [32]。SimPO 进一步采用 reference-free 奖励和显式 margin，并用平均 log probability 缓解长度伪影 [55]。

这些方法不应被理解成单一排行榜，而应被理解成工程权衡。一个后训练报告必须说明标签类型、是否使用 reference model、如何处理回答长度、偏好数据覆盖哪些安全类别，以及哪些独立评测防止 reward hacking。偏好目标很容易优化，但如果数据定义含糊，优

化结果也很难解释。

13.4 安全与 AI Feedback

Constitutional AI 和 RLAIIF 用规则或 AI 反馈减少人工标注负担 [6, 44]。但安全不是简单增加拒答数据。系统必须区分有害请求、合法敏感请求、误拒和越权工具行为。

13.5 深入展开：偏好学习优化的是代理目标

本章从一个基本事实开始：很多助手输出没有唯一正确答案。摘要可以有多个合理版本，代码补丁可以有不同维护性取舍，敏感问题需要在有用和安全之间权衡。偏好学习把“给定标准答案”改成“在同一 prompt 下比较多个候选回答”。这个比较依赖 rubric、标注人、候选生成分布和政策版本，因此不是宇宙真理。

Reward model 通常用 Bradley-Terry 目标训练，学习 prompt-response 的标量分数差。它只能在训练分布附近可靠；一旦策略优化生成出标注人没见过的样式，就可能出现 reward hacking。模型会学会冗长、免责声明、奉承、过度自信或某些标注捷径，让 reward 上升而真实质量下降。本章要求画出 reward 分数和独立人类评测随 checkpoint 的关系，而不是只看 reward。

PPO 式 RLHF 把策略、reference model、reward model 和 value head 放进一个昂贵循环。KL 约束防止策略偏离太远，但 β 过大限制提升，过小导致风格崩坏和 reward hacking。DPO 通过隐式 reward 推导，把偏好优化写成离线监督目标，省去在线 RL 和显式 reward model，但仍依赖偏好数据覆盖范围和 reference model。

DPO 之后的 KTO、ORPO、SimPO 等方法进一步探索是否需要成对比较和 reference model。KTO 使用 desirable/undesirable 标签；ORPO 把监督学习和 odds-ratio 惩罚结合；SimPO 使用 reference-free margin 和长度归一化。本章的判断是：这些不是万能优劣排序，而是后训练工程空间中的不同取舍。关键问题仍是数据定义了什么行为，独立评测能否发现它没有定义的失败。

13.6 章节细节

13.6.1 对齐作为偏好建模

许多助手任务没有唯一正确答案，因此后训练常用偏好比较来表达质量。偏好不是客观真理，而是 rubric、标注者、候选生成器和政策版本共同定义的局部信号。对齐研究必须承认这种代理性。

13.6.2 从标签到比较

偏好数据通常比较 chosen 和 rejected 回答。比较比单一评分稳定，但仍受候选质量影响。若候选都很差，chosen 只表示相对更好，不表示真正可发布。

13.6.3 采样候选回答

候选回答来自当前模型、旧模型、教师模型或不同采样温度。候选分布决定偏好数据覆盖哪些错误。只在容易样本上采样，会让后训练无法处理真实困难请求。

13.6.4 标注协议

标注协议应说明有用性、真实性、安全性、简洁性、引用和格式的权重。标注者需要处理冲突目标，并记录不确定性。没有协议，偏好数据只是不可解释的投票。

13.6.5 Bradley-Terry 目标

Reward model 常用 Bradley-Terry 形式，让 chosen 分数高于 rejected。它学习的是相对排序，不是绝对质量。分数校准和不确定性对后续 RL 很重要。

13.6.6 校准与不确定性

Reward model 在训练分布外可能自信错误。应检查分数分布、人类一致性、切片表现和不确定样本。校准差的 reward model 很容易在策略优化中被利用。

13.6.7 过优化

策略如果过度追求 reward，可能学会冗长、奉承、免责声明和格式捷径。Reward 上升不代表真实质量上升。独立人类评测和安全回归是发现 reward hacking 的必要手段。

13.6.8 带 PPO 的 RLHF

PPO 式 RLHF 使用策略、reference model、reward model 和 value head。KL 约束限制策略偏离 reference，平衡提升和稳定。它昂贵、复杂，但能直接优化序列级偏好。

13.6.9 PPO 机制

PPO 通过 clipped objective、优势估计和值函数训练稳定更新。实现中要监控 KL、reward、entropy、长度、拒答率和崩坏样例。小超参数变化会显著影响风格和安全边界。

13.6.10 系统成本

RLHF 需要在线采样、reward 打分、策略更新和大量日志。它比 SFT 或 DPO 更消耗算力和工程复杂度。成本应和实际质量收益一起评估。

13.6.11 DPO

DPO 把偏好优化写成离线监督目标，绕开显式 reward model 和在线 RL。它更简单、更稳定，但仍依赖偏好数据覆盖和 reference model。DPO 不是免评测的捷径。

13.6.12 取舍

PPO、DPO、KTO、ORPO、SimPO 等方法在数据需求、稳定性、成本和目标假设上不同。选择哪种方法应取决于任务、数据和系统约束。没有一种目标能替代独立评测。

13.6.13 DPO 之后的偏好目标

后续目标探索是否需要成对比较、reference model 或显式 margin。KTO 使用 desirable/undesirable 标签，ORPO 结合监督和 odds-ratio，SimPO 使用 reference-free margin。它们扩展了后训练工具箱，但没有改变偏好信号的局限。

13.6.14 安全、拒答与 AI Feedback

RLAIF 和 Constitutional AI 用规则或模型反馈减少人工标注成本。它们能扩展安全数据，但也可能继承规则盲点和模型偏见。安全训练应区分有害请求、合法敏感请求、拒拒和工具越权。

13.6.15 评测提醒

偏好分数不是地面真值。模型可能在标注分布上变好，却在真实用户、长任务或高风险领域退化。评测应包括人类偏好、事实性、安全、回归、成本和失败样例。

13.7 关键术语、实现要点与练习

关键术语。 Preference data 是比较或评分；reward model 学习代理偏好；KL regularization 限制策略偏离；DPO 是离线直接偏好目标；reference-free objective 不依赖冻结 reference model；reward hacking 是优化代理目标却损害真实目标。

实现要点。 偏好数据应记录 rubric、标注者、候选生成策略和政策版本；reward model 要看校准和切片；PPO 要监控 KL、reward、entropy、长度和拒答率；DPO、KTO、ORPO 和 SimPO 等直接偏好目标要监控长度、chosen/rejected log-probability gap 和回归。

练习。

- 1. 写出 Bradley-Terry pairwise loss 并解释 reward shift 不变性。
- 2. 比较 PPO 和 DPO 的系统成本。
- 3. 用同一偏好数据比较 DPO 和一个 reference-free 目标。
- 4. 设计 comply/refuse/redirect 安全边界评测。

13.8 结构化检查表

13.8.1 偏好学习失败诊断

失败模式	症状	诊断
Reward hacking	reward 上升，人评下降	画 reward-人评曲线，审查高 reward 样本
长度偏差	长回答无实质优势仍获胜	长度控制对比
过度拒答	合法敏感问题被拒	comply/refuse/redirect 边界集
安全泛化不足	改写后绕过拒答	多语言、工具、多模态红队
标注漂移	不同时期标签分布变化	gold examples、校准会议
Reward 过度自信	模糊样本 margin 很大	校准曲线、ensemble、abstention

第十四章 推理与测试时计算

14.1 推理是预算化计算

Chain-of-thought、自一致性、分解、程序化推理、树搜索和验证器都可以被看作测试时计算分配策略 [87, 85, 93]. 重要问题是：生成多少候选、用什么评分、花多少 token、何时调用工具、何时停止。

14.2 RLVR 与 GRPO

DeepSeek-R1 表明，基于可验证任务的大规模强化学习可以激发反思、验证和动态策略等推理行为 [17]. GRPO 用组内相对优势替代 learned critic，降低内存和复杂度，但依赖样本组多样性和奖励可靠性。RLVR 的关键风险是奖励捷径：模型可能学会通过格式或漏洞满足验证器，而不是真正稳健推理。

14.3 自适应测试时计算

测试时扩展研究正在从“多花 token 是否更强”转向“如何按难度分配预算”。综述把 test-time scaling 分解为 scale 什么、如何 scale、在哪里 scale、如何评估 [98]; 预算化推理进一步区分固定预算控制和按置信度/难度自适应分配 [3]。

14.4 深入展开：推理方法首先是预算分配方法

本章把 reasoning 看成测试时计算分配。Chain-of-thought 通过生成中间步骤给模型更多条件 token；self-consistency 采样多个推理路径再投票；decomposition 把复杂任务拆成子任务；program-of-thought 把部分推理交给代码解释器；tree search 和 MCTS 把候选扩展成搜索树；verifier 或 reward model 选择更可信结果。它们的共同点不是“更像人思考”，而是用更多计算换取更高成功率。

推理轨迹并不一定忠实。模型生成的 CoT 可能是解释性文本，而不是内部真实计算；隐藏或编辑轨迹也不一定改变答案。因此区分“有助于性能的外部工作区”和“可解释性证据”。如果轨迹被用于用户展示，还要考虑安全、隐私和误导风险；如果轨迹只用于内部选择，则要评估它是否真的提高正确率。

RLVR 和 GRPO 类方法把可验证奖励引入后训练。数学答案、代码单元测试、形式化证明和规则检查提供比人类偏好更便宜的奖励，但也会产生捷径。模型可能学会迎合验证器格式、利用测试漏洞或过度优化可验证任务，导致泛化不足。Process supervision 可以约束中间步骤，但标注成本高且定义困难。

预算控制是前沿推理模型的产品接口。一个系统可以给简单问题短答，给难题长推理，给代码任务调用测试，给事实问题检索，给高风险问题拒答或要求人工。本章要求每个推理方法报告成功率、token 成本、延迟、采样数、验证器质量、失败模式和 benchmark contamination 风险，而不是只报告最高分。

14.5 章节细节

14.5.1 推理作为预算化计算

推理方法可以理解为如何在测试时花额外计算。生成更多步骤、更多候选、更多工具调用或更多验证，都在改变预算。关键问题是额外计算是否带来可测收益。

14.5.2 Chain of Thought

Chain-of-thought 让模型生成中间步骤，常能提升数学和多步任务表现。它也可能产生不忠实或误导性解释。是否展示推理轨迹，需要考虑安全、隐私和用户理解。

14.5.3 Self-Consistency

Self-consistency 采样多个推理路径，再用投票或聚合选择答案。它用更多 token 换更高准确率。收益取决于候选多样性、错误相关性和聚合规则。

14.5.4 分解与程序化推理

复杂问题可以拆成子问题，或把计算交给程序执行。Program-of-thought、工具调用和代码解释器把部分推理外包给可验证系统。边界是工具权限、执行安全和错误传播。

14.5.5 Sample-and-Rank

Sample-and-rank 生成多个候选，再用 verifier、reward model 或规则排序。它适合有可比较候选的任务。排序器本身若不可靠，会选择看起来好但实际错误的答案。

14.5.6 结果监督与过程监督

Outcome supervision 只看最终答案，process supervision 评价中间步骤。过程监督能提供更密集信号，但标注成本高，且步骤是否忠实仍需研究。可验证任务常用最终答案奖励。

14.5.7 树搜索

Tree-of-thought 和 MCTS 把推理扩展为搜索问题。模型生成状态和动作，评分器决定扩展方向。搜索带来更强探索，也带来成本、剪枝和评估误差问题。

14.5.8 可验证奖励强化学习

RLVR 用数学答案、单元测试、规则检查或执行结果作为奖励。它减少主观偏好依赖，适合代码、数学和结构化任务。难点是可验证任务覆盖有限，模型可能过拟合验证器。

14.5.9 GRPO

GRPO 用组内相对优势替代显式 value model，降低训练复杂度。DeepSeek-R1 等工作显示大规模 RLVR 可以激发推理行为。仍需评估分布外任务、轨迹质量和安全边界。

14.5.10 推理时扩展

Inference-time scaling 研究花更多测试时计算如何提升能力。可以扩展 token、样本、搜索深度、工具调用或 verifier 预算。评价时应报告成本曲线，而不是只给最贵设置下的分数。

14.5.11 预算强制

预算强制让模型在固定 token 或步骤内完成任务，或按难度自适应延长。它把推理能力和资源管理联系起来。真实产品需要在质量、延迟和成本之间选择。

14.5.12 计算最优分配

不同问题需要不同预算。简单问题多采样会浪费，困难问题预算不足会失败。自适应策略应估计难度和置信度，并有停止规则。

14.5.13 前沿推理系统

前沿系统把预训练、SFT、RLVR、工具、搜索和服务调度结合起来。o 系列模型以及 DeepSeek-R1、Qwen3、Kimi K2 等系统报告，都把推理预算作为能力组成。研究需要把模型和推理控制器一起评测。

14.5.14 答案准确率不够

只看最终答案会掩盖偶然猜对、工具误用、不可解释成本和安全问题。推理评测还应看步骤、成本、稳定性、拒答、校准和失败类型。尤其在高风险任务中，过程证据很重要。

14.5.15 轨迹忠实性

模型给出的推理文字未必是内部真实原因。轨迹可能是事后解释，也可能被训练成符合人类期望。把轨迹当作检查证据前，应先验证其忠实性。

14.5.16 推理边界测试

边界测试包括干扰信息、错误前提、反事实、长链条、多解问题和不可解问题。强模型应知道何时继续推理，何时停止，何时拒绝。推理越强，越需要避免自信地扩大错误。

14.6 关键术语、实现要点与练习

关键术语。 Chain-of-thought 生成中间步骤；self-consistency 多样采样再选择；verifier 评价候选；RLVR 用可验证奖励；GRPO 用组内相对优势；budget forcing 控制推理长度。

实现要点。 推理方法必须报告生成候选、评分器、预算、延迟、失败模式和污染风险；轨迹不等于忠实解释；可验证奖励要防止格式捷径和测试漏洞；自适应预算应按难度和置信度分配。

练习。

1. 比较 single-shot、CoT、自一致性和工具执行的成本。
2. 为数学任务设计 verifier，并列可能被利用的漏洞。

- 3. 设计一个按难度增加采样数的自适应策略。
- 4. 解释为什么 pass@k 不等于部署中的单次可靠性。

14.7 结构化检查表

14.7.1 推理方法比较

方法	增加的计算	主要风险
CoT	中间 token	轨迹不忠实、泄露敏感推理
Self-consistency	多样采样	成本高、选择偏差
Program-of-thought	解释器执行	代码安全、环境依赖
Verifier	候选评分	验证器漏洞、reward hacking
Tree search	扩展多个状态	状态爆炸、启发式偏差
RLVR/GRPO	后训练采样与奖励	奖励捷径、任务覆盖窄

第十五章 多模态与生成式基础模型

15.1 模态作为接口

多模态模型把图像、视频、音频、文档和语音接入语言模型。图像、音频、视频和传感器流并不是天然的文本 token 序列，因此多模态模型首先要解决接口问题：如何把连续信号转成语言模型可以使用的表示，同时保留任务所需的信息。CLIP 通过图文对比学习建立联合表示 [68]；Flamingo、BLIP-2、LLaVA 等系统展示了冻结视觉编码器、投影层、query transformer 和跨注意力等连接方式 [2, 46, 49]。

早期 VLM 常被理解为“给语言模型接一个视觉编码器”：编码器把图像变成 patch 或 embedding，connector 把这些表示映射到语言模型隐藏空间，语言模型再完成问答、解释或结构化输出。这个设计简单有效，但也继承了两端的弱点：视觉编码器可能忽略小字和空间关系，语言模型可能用文本先验补全并不存在的视觉事实，connector 可能为了省 token 丢掉细节。因此，多模态能力不能只看回答是否流畅，还要看它是否真正读到了图像、图表、页面或视频片段。

15.2 统一理解与生成

现在的前沿不再只是“看图回答问题”。理解模型和生成模型正在合流：一个系统可能既要读图、读文档、听语音、理解视频，又要生成图像、编辑图像、合成语音或生成视频。统一多模态理解与生成综述把相关工作分成 autoregressive、diffusion-based 和 hybrid 三类，并指出 tokenization、跨模态注意力、数据平衡和 benchmark 是共同难点 [100]。

统一模型同时面对两个方向相反的任务。理解要求忠实抽取：读清输入、绑定实体、保留空间和时间证据，并在证据不足时拒答。生成要求可控合成：创造新的媒体，遵循指令，保持一致性，并满足安全和来源要求。只用 VQA 准确率或图像审美分数评价这样的系统是不够的；它需要同时报告感知、推理、编辑、可控性、多样性、延迟和误用风险。

Janus-Pro 展示了自回归统一的一条路线：它在多模态理解和 text-to-image instruction following 上同时改进，并强调训练策略、数据规模、模型规模和生成稳定性 [13]。这类工作

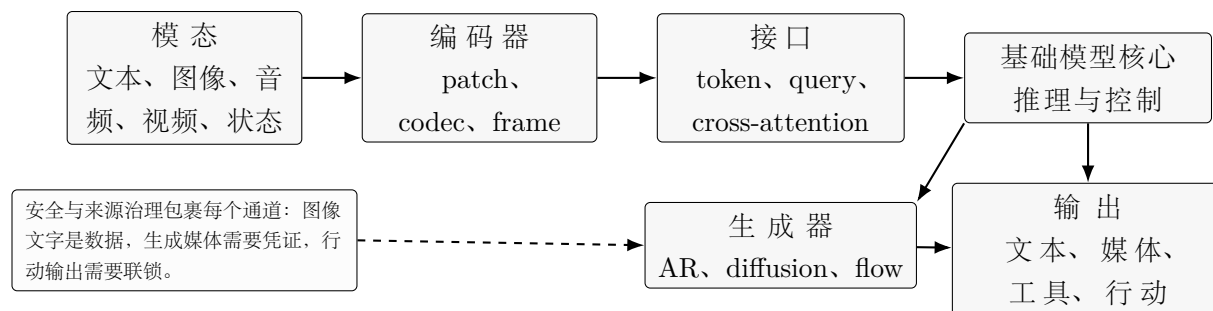


图 15.1: 统一多模态系统首先是接口设计问题。共享核心模型不意味着各模态可以共享同一套成本、评测和安全边界。

说明，理解图像的表达不一定最适合生成图像；统一系统常常需要共享高层语义推理，同时在高保真生成处保留模态专门接口。

15.3 视频、音频与 Omni 模型

视频引入时间维度，音频引入采样、说话人、韵律和流式延迟。Qwen3-VL 报告了长上下文多模态输入、视频时间对齐和视觉推理能力 [67]。Qwen3-Omni 进一步把文本、图像、音频、视频理解和语音生成放进统一系统，并强调低延迟流式输出 [90]。

Omni 模型把接口问题推到更极端的位置。用户可能输入文本、截图、照片、视频、语音或它们的组合，并期待系统输出文本、语音、图像或行动。AR-Omni 代表一种更彻底的自回归方案：把文本生成、图像生成和流式语音生成放在单一 Transformer decoder 下，用统一 token 流处理任意到任意生成 [14]。这样的方案是否长期占优，不取决于概念是否优雅，而取决于它能否同时处理模态不平衡、实时性、视觉保真、语音自然度、安全策略和服务成本。

15.4 生成模型目标

自回归语言建模只是生成目标的一种。文本天然序列，因此 next-token factorization 很自然；但图像、音频和视频可以被序列化，也可以通过扩散、流匹配或混合目标建模。Diffusion Transformer 用 transformer 替代传统 U-Net denoiser，在 latent patch 上做图像生成，并显示了可扩展性 [64]。Sora 把视觉数据压缩成 latent，再切成 spacetime patches，用 text-conditioned diffusion model 生成图像和视频 [60]。Stable Diffusion 3 相关工作使用 rectified flow transformer 进行高分辨率文本到图像合成，并强调文本 token 与图像 token 的双向交互 [20]。

目标族	适合的问题	成本表面	常见失败
Autoregressive	文本、代码、语音 token、工具轨迹	序列 decode 和 KV cache	空间顺序别扭, 高分辨率媒体慢
Diffusion	图像/视频质量、编辑、多样性	多步去噪和 guidance	prompt 漂移, 采样贵, 依赖过滤器
Rectified flow	噪声到数据的连续流	步数、latent 分辨率	条件对齐弱时生成不忠实
Hybrid AR-diffusion	语义规划加连续合成	级联系统延迟	媒体漂亮但不服从指令

扩散模型常用去噪目标:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{x_0, \epsilon, t, c} [\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2].$$

Rectified flow 则学习从噪声端点到数据端点的速度场。若噪声端点为 ξ_0 、数据端点为 x_1 , 且 $x_t = (1 - t)\xi_0 + tx_1$, 则

$$\mathcal{L}_{\text{flow}} = \mathbb{E}_{\xi_0, x_1, t, c} [\|v_\theta(x_t, t, c) - (x_1 - \xi_0)\|_2^2].$$

对本书而言, 关键不是要求读者都去实现视频生成器, 而是要理解: token 预测不是基础模型的全部。Dream 7B 这类离散扩散语言模型通过迭代去噪生成文本, 提供任意顺序生成、填充和质量-速度权衡 [95]. MMaDA 则把统一扩散形式用于文本推理、多模态理解和文生图, 并设计面向扩散基础模型的后训练方法 [92]. MammothModa2 代表混合 AR-Diffusion 路线: 自回归路径做语义规划, 扩散/flow 路径做高保真图像生成与编辑 [76]. 因此, 高水平教材不能只讲 decoder-only LLM, 还必须解释生成目标、模态表示和服务接口如何共同塑造系统能力。

15.5 行动、具身与世界模型

文本、图像、音频和视频之后, 下一个边界是行动。Vision-language-action (VLA) 模型把视觉观察和语言指令映射为动作, 通常把机器人动作表示为 token, 或者在多模态 backbone 后接动作头。RT-2 展示了一种直接路线: 把视觉语言模型同时在网页规模视觉语言任务和机器人轨迹上 co-finetune, 并把动作写成类似文本 token 的格式, 使模型能把部分网页语义知识迁移到机器人控制中 [10]. OpenVLA 则把这条路线做成更开放的版本: 一个 7B 开源 VLA, 在真实机器人 demonstration 上训练, 并提供面向新任务的微调和量化路径 [39].

VLA 让“基础模型”这个词变得更具体，也更危险。文本生成错误可以编辑；机器人动作会改变世界。一个 VLA 报告必须说明观察频率、动作表示、控制时域、延迟、机器人形态、重置策略、安全联锁、teleoperation 数据、sim-to-real 迁移和失败恢复。一个模型能在静态图像中识别杯子，并不代表它能在遮挡、摩擦、相机标定误差和执行器限制下稳定抓起杯子。

VLA 行动模型的行为克隆目标可写为

$$\mathcal{L}_{\text{VLA}} = - \sum_t \log \pi_{\theta}(a_t \mid o_{\leq t}, u, h_t),$$

其中 $o_{\leq t}$ 是观察历史， u 是语言指令， h_t 是可选任务记忆， a_t 是离散动作 token 或连续控制目标。若改用强化学习，则还要面对真实世界探索成本、安全联锁和重置代价。

VLA 综述把这个方向定义为视觉感知、自然语言理解和具身控制的统一，并强调数据集、仿真平台、架构范式和真实部署挑战 [84]。对本书而言，合理范围不是展开成完整机器人教材，而是把行动视为另一种更严格的模态：它有更强的时序、安全和评测契约。世界模型也在这里连接生成与行动：视频生成器预测未来画面，具身模型想象抓取后果，规划器模拟工具调用结果，本质上都在用生成模型辅助决策。风险在于把视觉可信误认为物理真实；生成未来看起来连贯，并不代表它满足动力学、接触、物体恒常性和任务约束。

15.6 多模态评测与安全

多模态评测应区分感知错误和推理错误。MMMU、MathVista、POPE、HallusionBench 和 MM-SafetyBench 分别覆盖专家知识、视觉数学、幻觉和安全风险 [96, 53, 47, 27, 50]。视觉 prompt injection、截图隐私、OCR 错误和敏感属性推断都需要单独测试。

生成式多模态系统还要测试指令遵循、视觉保真、文字排版、对象关系、时间一致性、编辑局部性、多样性、安全过滤和来源标记。一个统一理解-生成模型不能只在生成任务上变强，却默默降低 OCR、文档问答、语音延迟或拒答边界。评测报告必须说明是否使用外部 OCR、ASR、对象检测、检索或浏览工具；否则我们无法判断分数来自模型本身，还是来自系统管线。

15.7 深入展开：多模态和生成模型改变接口边界

本章已经从“多模态 LLM”扩展为“多模态与生成式基础模型”。传统 VLM 关注把图像接入语言模型：视觉编码器提取 patch 或 frame 特征，connector 映射到语言模型空间，语言模型生成文本。Projection、Q-Former、cross-attention 和 end-to-end training 各有成本和风险。文档、图表和 OCR 任务尤其会暴露 token 压缩和分辨率限制。

统一理解与生成进一步打破边界。Janus-Pro 说明理解图像和生成图像可能需要不同表示；MMaDA 和 Dream 7B 表明扩散形式也可以用于文本和多模态；Sora、DiT 和 Rectified Flow 说明高维视觉生成并不一定服从 next-token 预测；AR-Omni 和 Omni 系统尝试把文本、图像、音频、视频和语音放进统一接口。本章把这些方法放进同一问题：选择什么表示、什么目标函数、什么服务接口、什么评测证据。

行动模型把多模态推向现实环境。RT-2 和 OpenVLA 将视觉语言模型连接到机器人动作，说明动作也可以被 token 化或由多模态 backbone 产生。但机器人动作不是文本输出：它有控制频率、执行器限制、延迟、碰撞、重置、真实失败和人类安全问题。VLA 评测必须报告任务成功、危险动作、人类干预、sim-to-real gap 和恢复策略。

视频和世界模型则连接生成与预测。一个视频生成器可以产生看似合理的未来，但物理一致性、对象恒常性和因果关系可能错误。世界模型如果用于规划，必须证明它的预测能改善真实决策，而不是只生成好看的画面。因此把 temporal grounding、physical consistency、content provenance 和 safety filtering 放进同一章讨论。

15.8 章节细节

15.8.1 模态作为接口

模态不是附加功能，而是模型输入输出接口的扩展。图像、音频、视频、语音和动作都需要被编码成模型可处理的表示。统一接口的目标是让不同模态在同一任务语境中协作。

15.8.2 图文对齐

CLIP 类对比学习把图像和文本拉到共同空间，使零样本分类和检索成为可能。它学习的是跨模态相似度，不是完整生成能力。对比预训练常作为多模态系统的视觉基础。

15.8.3 零样本分类

零样本分类把类别写成文本提示，比较图像和文本 embedding。它显示自然语言可以作为视觉任务接口。局限是提示敏感、类别粒度和数据偏差。

15.8.4 冻结编码器与投影层

许多 VLM 冻结视觉编码器和语言模型，只训练投影层把视觉特征接到文本 token 空间。这种方法便宜、稳定，但视觉信息压缩会限制细粒度感知。是否端到端训练取决于数据和算力。

15.8.5 Query Transformer 与 Token 压缩

Q-Former 或 resampler 用少量查询 token 压缩视觉特征，减少 LLM 上下文负担。压缩提高效率，但可能丢失细节。OCR、图表和文档任务常常需要更高分辨率或专门结构。

15.8.6 Cross-Attention 与交错媒体

Cross-attention 让文本层直接读取视觉特征，交错媒体让图文序列混合出现。它们更灵活，也更复杂。训练数据必须覆盖多轮、多图、视频帧和指代关系。

15.8.7 架构取舍

多模态架构在冻结与端到端、投影与 cross-attention、视觉 token 数与成本、统一模型与专用模块之间取舍。没有一种结构适合所有任务。评测应覆盖感知、推理、生成和安全。

15.8.8 统一理解与生成

统一理解-生成模型试图同时处理识别、问答、编辑和生成。它们需要把离散语言 token、连续视觉特征和生成模型 latent 协调起来。关键挑战是接口统一、训练稳定和评测一致。

15.8.9 生成建模目标

多模态生成可用自回归、扩散、flow matching 或混合目标。自回归适合离散序列，扩散和流匹配常用于图像、音频和视频。目标函数决定采样过程、可控性和服务成本。

15.8.10 自回归生成

自回归图像或视频生成把视觉表示离散化或按序列预测。它与语言模型接口相似，但序列更长、局部相关更强。成本和长程一致性是主要挑战。

15.8.11 扩散与 Rectified Flow

扩散模型从噪声逐步去噪，rectified flow 学习更直接的流路径。它们在图像和视频生成中非常重要。采样步数、条件控制和安全过滤直接影响服务体验。

15.8.12 混合 AR-Diffusion 系统

许多系统用 LLM 处理指令和规划，再用扩散或流模型生成视觉内容。LLM 提供语义控制，生成模型提供高质量像素。系统评测必须看文本遵循、视觉质量、一致性和安全。

15.8.13 行动、具身与世界模型

Vision-language-action 模型把感知、语言和动作连接起来。它们需要处理实时观察、动作空间、环境反馈和安全约束。世界模型试图学习环境动态，但不能只用视频逼真度评估。

15.8.14 多模态指令微调

多模态 SFT 把图像、视频、音频和文本对话组织成统一模板。训练阶段通常包括连接器预训练、指令微调和偏好/安全调优。数据质量决定模型是否真正理解视觉内容。

15.8.15 训练阶段

第一阶段常让视觉特征对齐语言空间，第二阶段学习多模态指令，后续阶段优化偏好、安全和工具使用。每一阶段的冻结策略、学习率和数据混合都不同。跳过阶段可能降低稳定性。

15.8.16 数据来源

数据来自图文对、caption、OCR、VQA、文档、图表、视频问答、语音和合成数据。不同来源带来不同偏差和许可证问题。敏感图像、人脸、位置和医疗图像尤其需要治理。

15.8.17 带图像的 Chat Template

多模态模板要标明图像位置、图像数量、帧顺序、工具结果和回答边界。图像占位符不是普通词，它连接外部编码器输出。模板不一致会导致模型忽略图像或混淆多图指代。

15.8.18 OCR、图表与文档

OCR 和文档理解要求高分辨率、布局感知和精确引用。图表任务需要读数值、坐标轴、图例和趋势。普通图像问答能力不足以证明模型能处理业务文档。

15.8.19 系统成本

视觉 token、视频帧、音频片段和生成步数会显著增加成本。多模态服务的延迟包括编码、LLM prefill、decode、生成器采样和后处理。成本报告应按模态拆分。

15.8.20 视觉 Token 与 Prefill

一张高分辨率图像可能转成大量视觉 token，拉高 prefill 时间和 KV cache。压缩 token 可以降成本，但可能损失细节。系统需要按任务动态选择分辨率和帧数。

15.8.21 视频与音频

视频加入时间维度，音频加入连续信号和实时性要求。帧采样、时间对齐、说话人分离和延迟预算都会影响质量。实时语音对话还要处理打断、流式 ASR 和 TTS。

15.8.22 Benchmark 家族

多模态 benchmark 覆盖 VQA、OCR、图表、文档、视频、空间推理和生成质量。很多 benchmark 容易被污染或饱和。真实任务评测应加入私有文档、交互和错误审查。

15.8.23 评测提醒

多模态模型可能用语言先验猜答案，而不看图像。评测应包含反事实图像、文字覆盖、干扰对象、多图引用和需要精确读数的样例。生成模型还要评估可控性、一致性和安全。

15.8.24 安全与治理

多模态系统会产生隐私、身份、地点、医学、版权和视觉 prompt injection 风险。输入图像可能携带恶意指令，输出图像可能侵犯权利或造成误导。治理必须覆盖输入、模型、工具和生成内容。

15.8.25 视觉 Prompt Injection

视觉 prompt injection 把恶意指令嵌入图片、网页截图、文档扫描或视频帧中，诱导模型忽略系统策略、泄露上下文或越权调用工具。防御不能只依赖提示词警告，而应在架构上把图像和检索文档都视为不可信数据；工具权限、密钥访问、外部请求和文件写入必须由运行时强制控制。

15.8.26 隐私与敏感推断

多模态模型可能从人脸、地理位置、医疗影像、身份证件、屏幕截图和家庭环境中推断敏感信息。即使用户没有显式询问，模型也可能在描述中暴露身份、健康、位置或财务线索。发布系统需要输入最小化、日志脱敏、访问控制、保留期限和敏感类别拒答策略。

15.8.27 幻觉与 Grounding

多模态幻觉表现为看见不存在的物体、误读图表数值、把语言先验当作视觉证据，或在视频中错误归因动作。Grounding 评测应要求模型把回答连接到图像区域、文档文本、帧时间或引用证据。对 OCR、图表、医学和遥感等场景，流畅描述不能替代可核查定位。

15.9 关键术语、实现要点与练习

关键术语。 Vision encoder 把图像或视频映射成特征；connector 把模态特征接入 LLM；unified understanding-generation model 同时理解和生成媒体；diffusion model 通过去噪生成；VLA model 根据观察和语言产生动作；world model 预测未来状态或行动后果。

实现要点。多模态报告应说明分辨率、视觉 token 数、帧采样、OCR/ASR 工具、encoder latency、prefill/decode 成本和安全过滤；生成模型应说明 objective、latent/token 表示、guidance、step schedule 和 provenance；VLA 应说明动作空间、控制频率和安全联锁。

练习。

1. 比较 projection、Q-Former 和 cross-attention connector。
2. 设计一个文档 VQA 评测，区分 OCR、布局和推理错误。
3. 比较 autoregressive、diffusion 和 AR-diffusion 的生成取舍。
4. 为桌面机器人设计 VLA 评测，包含 reset、危险动作和人类干预。

15.10 结构化检查表

15.10.1 多模态系统报告清单

字段	内容
输入处理	分辨率、裁剪、帧采样、音频切片、OCR/ASR 工具
表示接口	vision encoder、connector、visual tokens、image marker、position policy
生成目标	autoregressive、diffusion、rectified flow、hybrid AR-diffusion
服务成本	encoder latency、prefill、decode、生成步数、显存、流式延迟
评测切片	感知、OCR、图表、空间关系、视频时间、生成质量、安全
来源治理	metadata、watermark、content credentials、过滤器、日志
行动系统	observation、action space、control frequency、reset、安全联锁

第十六章 评测、安全与治理

16.1 评测是测量系统

评测从问题开始：要支持什么决策？客服助手、代码 agent、医疗信息助手和教育系统需要不同指标。MMLU、HELM、GPQA、SWE-bench、HLE 和 BrowseComp 提供公共参照，但每个 benchmark 都有观念边界 [30, 48, 71, 36, 65, 61]。

长上下文、Agent 和生成式多模态系统会在传统短问答 benchmark 之外失败。长上下文评测应测试有效上下文利用，而不是只看最大输入长度；需要报告位置敏感性、干扰鲁棒性、答案证据和每个成功任务的成本 [99, 33, 8]。Agent 评测还要把模型、工具运行时、状态管理、重试策略、沙盒和预算分开看。SWE-bench 和 BrowseComp 这类评测之所以重要，是因为它们测量的不是单句语言能力，而是一个模型-工具-环境组合在约束下完成任务的能力 [36, 61]。

生成式多模态评测需要另一套证据。文生图或文生视频模型应评估指令遵循、视觉质量、文字排版、对象关系、时间一致性、编辑局部性、多样性、安全过滤和来源标记。统一理解-生成系统还需要跨任务回归检查：提升图像生成不能以牺牲 OCR、文档问答、语音延迟或安全拒答为代价。Janus-Pro、MMaDA、Sora 风格视频生成和 AR-Omni 都说明，一个平均分无法代表既能理解又能创造媒体的系统 [13, 92, 60, 14]。

视频尤其需要单独评测。VBench++ 把视频生成质量拆成 temporal flicker、subject consistency、motion smoothness、spatial relationship、text-to-video、image-to-video 和 trustworthiness 等细粒度维度 [35]。T2VPhysBench 进一步追问生成视频是否遵守基本物理规律，而不只是看起来漂亮 [28]。视频理解也有类似问题：TOC-Bench 专门评估 temporal object consistency，要求模型在遮挡、消失、再出现、状态变化和对象交互中保持同一对象的身份和状态 [12]。因此，一个视频系统可能画面质量很高、物理一致性很弱，同时在对象中心时序推理上仍然失败。

具身和行动系统还要承担真实世界责任。VLA 评测应报告任务成功率、失败恢复、危险动作率、人类干预率、延迟、跨机器人形态迁移，以及测试任务是否和训练数据共享物体、场景、操作者或 teleoperation policy。仿真可以加速评测，但如果不测量 sim-to-real gap，就不能替代真实部署证据。机器人场景中，即使平均失败率低，罕见失败也可能造成

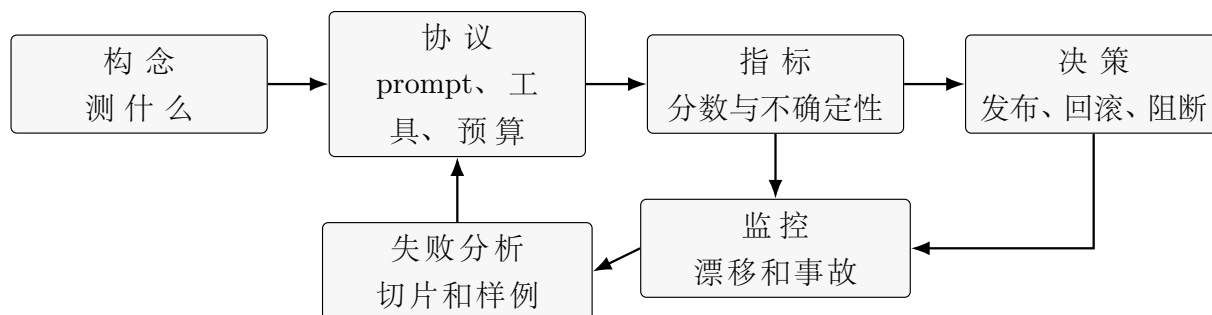


图 16.1: 评测是发布证据闭环。没有协议和不确定性的分数，不能支撑系统发布。

财产损失或人身风险。

16.2 人类评测与 LLM-as-Judge

人类评测需要明确 rubric、标注人资格、一致性、置信区间和失败样例。LLM-as-Judge 可以扩大规模，但会继承 judge 模型偏差、位置偏差、长度偏差和脆弱性 [102, 51]。高风险发布不能只依赖模型裁判。

16.3 治理

治理包括模型卡、数据卡、系统卡、风险框架、红队、监控、事件响应和回滚。NIST AI RMF、NIST GenAI Profile、EU AI Act、OpenAI Preparedness Framework 和 Anthropic Responsible Scaling Policy 都说明：前沿模型发布需要把能力、风险、缓解措施和残余不确定性写成可检查文档 [58, 59, 22, 62, 5]。

16.4 可解释性、遗忘与水印

治理需要技术抓手，但任何抓手都不能被夸大。机制可解释性尝试识别模型内部的特征、回路或因果路径。Anthropic 关于 monosemantic features 和 circuit tracing 的工作表明，较大模型中的某些计算结构可以被部分解释，但这仍然不是完整安全证明 [81, 4]。可解释性证据必须和干预实验、反例和下游行为测试一起使用。

Unlearning 关注另一个问题：模型训练后，如何让它停止产生或依赖特定知识、隐私数据、版权材料或危险能力。LLM unlearning 综述强调，删除声明必须同时评估保留能力、抽取抵抗、重新学习风险和连带损伤 [66]。一个模型不再回答某个记忆化 prompt，并不表示它不会通过改写、检索或微调泄露同一信息。

水印和来源工具试图识别生成内容或追踪模型输出。文本水印可以通过改变采样分布留下可检测统计信号 [41]。对多模态生成而言，来源还包括元数据、内容凭证、可见标记、模型卡和平台政策。水印可能被改写、裁剪、转码或分布漂移破坏，所以它只能作为证据层之一，而不是解决虚假信息、版权或问责问题的完整方案。

合成内容透明度需要分层处理。NIST 关于 synthetic content transparency 的报告把认证、来源追踪、标签、水印、检测、测试和检查视为互补技术，而不是单一解决方案 [11]。C2PA 规范则提供了内容凭证和媒体来源历史的技术标准，用签名元数据记录资产来源、编辑和生成过程 [15]。这些工具必要但有限：元数据可能被剥离，视觉水印可能被裁剪，统计水印可能被改写削弱，生成资产被多次 remix 后来源链也可能变得含糊。治理策略应把模型侧水印、签名内容凭证、可见标签、平台检测、用户举报和事故响应组合起来，并明确当来源信号缺失或冲突时系统如何处理。

16.5 深入展开：评测是发布证据

本章把评测、安全和治理合并，是因为它们在发布决策中不可分割。一个 evaluation card 应说明 construct、population、protocol、metric、uncertainty 和 decision rule。MMLU、HELM、GPQA、SWE-bench、HLE 和 BrowseComp 都只是特定构念的测量工具，不能直接等同于“模型好”。

人类评测需要 rubric、标注者资格、校准、一致性和置信区间。LLM-as-judge 可以降低成本，但有位置偏差、长度偏差、自我偏好、风格偏差和政策偏差。本章建议用 answer-order randomization、长度控制、judge ensemble、人类校准和 adversarial judge tests 缓解，但高风险发布不能只依赖模型裁判。

长上下文、agent 和生成式系统需要额外协议。长上下文要测位置敏感性、聚合、多跳、矛盾和 distractor；agent 要测工具权限、重试、环境状态和 rollback；视频生成要测时序一致性、物理一致性和对象身份；VLA 要测危险动作率和人类干预。一个平均分无法覆盖这些系统级行为。

治理把评测变成可检查 artifact。Model cards、datasheets、system cards、NIST AI RMF、EU AI Act、Preparedness Framework 和 RSP 都在要求团队说明用途、限制、数据、风险、缓解、监控和回滚。可解释性、unlearning、水印和内容凭证都是技术证据层，但都不是单独充分条件。发布决策必须承认不确定性，并说明事故发生时如何发现、响应和撤回。

16.6 章节细节

16.6.1 评测作为测量系统

评测是把问题、样本、指标、评分器和决策连接起来的测量系统。一个分数只有在知道测量对象和误差来源时才有意义。发布决策需要证据包，而不是单个排行榜数字。

16.6.2 从问题到决策

评测前应先明确要支持什么决策：是否训练继续、是否发布、是否替换模型、是否进入高风险场景。不同决策需要不同严格度。研究 benchmark 和生产发布评测不能混用。

16.6.3 测量模板

测量模板应记录任务定义、样本来源、时间范围、切分方法、评分规则、置信区间、失败分类和复现脚本。没有模板，评测结果难以比较。模板也能防止事后挑指标。

16.6.4 能力 Benchmark

能力 benchmark 覆盖语言、数学、代码、知识、长上下文和多模态。它们适合快速比较，但容易被污染、饱和和针对性优化。高分应被视为线索，而不是部署证明。

16.6.5 pass@k 与选择效应

pass@k 衡量多次采样中至少一次成功的概率，常用于代码和数学。它不能代表单次用户体验，也可能被大量采样堆高。报告时应同时给出采样次数、成本和选择规则。

16.6.6 Benchmark 污染

污染会让模型在测试题上表现虚高。来源包括训练语料、论坛解析、翻译版本和合成数据。污染检查应成为数据报告和模型卡的一部分。

16.6.7 Benchmark 饱和与 Gaming

热门 benchmark 会被反复调参，逐渐失去区分度。模型可能学会题型和评分器偏好，而不是提升真实能力。评测套件应定期更新，并包含隐藏或动态样本。

16.6.8 长上下文、Agent 与生成评测

长上下文评测要看证据定位、整合和抗干扰；agent 评测要看任务完成、工具成本和权限；生成评测要看质量、可控性和安全。不同系统形态需要不同指标。统一平均分会掩盖关键失败。

16.6.9 Rubric 与成对偏好

人类评测需要明确 rubric，说明真实性、有用性、安全性、简洁性和风格如何权衡。成对偏好比绝对评分稳定，但仍依赖候选分布。标注一致性和争议样例应被报告。

16.6.10 LLM-as-Judge

LLM-as-judge 能降低评测成本，但会继承模型偏见、位置偏好和风格偏好。它适合作为辅助，不应无校准地替代人类和客观指标。应对 judge 做一致性、盲测和对抗检查。

16.6.11 真实性与事实精度

事实性评测要区分可验证事实、开放解释和主观建议。RAG 系统还要检查回答是否被引用证据支持。模型说得流畅不等于事实正确。

16.6.12 校准与弃答

可靠系统应知道何时不确定。校准评测比较置信度和正确率，弃答评测检查模型是否在证据不足时拒绝。过度自信和过度拒答都是质量问题。

16.6.13 鲁棒性

鲁棒性包括提示改写、噪声、对抗指令、分布偏移、长上下文干扰和工具错误。模型在标准样本上正确，不代表在真实输入中稳定。鲁棒性评测应覆盖常见用户错误和恶意输入。

16.6.14 政策分类

安全评测需要明确政策 taxonomy，例如自伤、暴力、网络、隐私、医疗、金融和违法行为。不同类别有不同拒答和安全替代策略。模糊政策会让标注和模型都不稳定。

16.6.15 红队

红队通过主动寻找失败来补充常规评测。它应记录攻击路径、成功条件、复现步骤和修复验证。红队不是一次性活动，而是发布和更新周期的一部分。

16.6.16 文档

模型卡、系统卡、数据说明和评测报告让外部读者理解模型边界。文档应说明训练数据、能力、限制、安全评测、适用场景和禁止场景。没有文档，治理无法落地。

16.6.17 风险管理框架

NIST AI RMF、EU AI Act 等框架把风险识别、测量、管理和治理制度化。它们不能替代技术评测，但能提供责任结构。高风险部署需要把技术证据映射到合规要求。

16.6.18 可解释性、遗忘与水印

可解释性帮助理解模型机制和失败；machine unlearning 试图移除特定信息影响；水印试图标识生成内容。它们都不是万能解决方案。应报告适用条件、绕过方式和残余风险。

16.6.19 运行监控

发布后仍要监控输入分布、拒答率、错误、延迟、成本、安全事件和用户反馈。模型环境会变化，静态评测会过期。监控和回滚是治理的一部分。

16.6.20 部署限制

某些场景需要人类审核、权限限制、输出水印、引用要求或禁止自动决策。部署限制不是失败，而是风险控制。高能力模型尤其需要明确边界。

16.7 关键术语、实现要点与练习

关键术语。 Construct 是评测意图测量的能力；benchmark contamination 是测试信息泄漏；LLM-as-judge 用模型裁判输出；red teaming 主动寻找失败；unlearning 试图删除知识或能力；content credentials 是签名来源元数据。

实现要点。 Evaluation card 应包含 construct、population、protocol、metric、uncertainty 和 decision rule；安全评测要分 unsafe compliance 和 false refusal；生成式系统要评测 provenance；治理文档要连接模型、数据、工具、监控和回滚。

练习。

- 1. 为一个客服 RAG 助手写 evaluation card。
- 2. 设计 LLM-as-judge prompt，并列出五种 judge bias。
- 3. 为文生视频模型设计视觉质量、时序一致性、物理一致性和来源评测。
- 4. 为 VLA 发布写安全 checklist。

16.8 结构化检查表

16.8.1 评测与治理发布清单

领域	问题	证据
任务成功	是否解决用户问题	in-domain set、人类任务审查、工具日志
事实性	声明是否被支持	atomic fact、RAG faithfulness、abstention
安全	是否拒绝正确请求	unsafe compliance、false refusal、red team
鲁棒性	输入扰动是否破坏行为	改写、对抗、长上下文、工具失败
成本	是否可部署	延迟、token、硬件、美元/任务
治理	是否可检查	model card、data doc、policy version、rollback
来源	生成内容是否可追踪	watermark、C2PA、metadata、检测、用户举报

第十七章 研究前沿与实践路线

17.1 为什么前沿实践会改变课程

前面各章覆盖了 大语言模型与生成式基础模型的主要生命周期，但前沿变化太快，不能只靠固定目录。最新系统常常在章节之间产生创新：架构变化依赖数据和训练稳定性；推理模型依赖奖励、采样和验证；多模态模型依赖服务延迟和安全策略；生成模型依赖目标函数、token/latent 表示和安全来源标记；agent 能力依赖工具协议和评测环境。

CS336 的价值在于强调从零实现：tokenizer、模型、优化器、GPU kernel、并行、scaling laws、数据处理、评测和 alignment/reasoning RL 都要亲自构建或测量 [78]。这也是本书建议的学习方式。

17.2 路线图范围

结合 CS336 课程结构和 2025–2026 年前沿论文，本书当前 17 章结构仍然合理。把 LLM、多模态理解、图像/视频生成、具身 agent 和治理拆成多个孤立章节，表面上会增加覆盖面，但会削弱本书最重要的教材论证：现代基础模型是一条耦合的生命周期，而不是论文主题的并列清单。因此，本书采用的策略不是继续增加许多短章，而是扩大第 15 章为“多模态与生成式基础模型”，在第 12 章补上下文工程和记忆，在第 13 章补 DPO 之后的偏好目标，在第 16 章补长上下文、agentic、生成式、具身行动评测以及可解释性、遗忘、水印和来源治理。

这个结构的好处是主线清楚：前两章定义数据和 token；中间章节解释 Transformer、训练、系统和服务；后训练章节解释如何把基础模型变成助手；应用章节解释检索、工具、agent、推理、多模态和生成；最后用评测和治理把所有能力放回可发布系统。高水平书稿需要的不是追逐每个模型名，而是给读者稳定的分析框架。

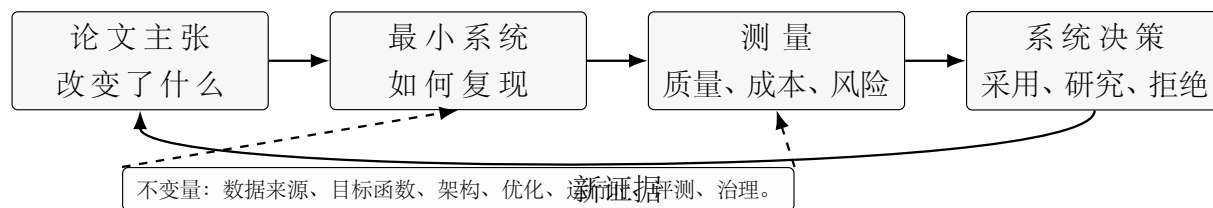


图 17.1: 前沿实践从论文主张走向最小复现、测量和系统决策。模型名称会变化，证据闭环不应变化。

17.3 前沿扩展方向

第一，实验作业应继续增加。读者应写 tokenizer、小 GPT、SFT collator、LoRA adapter、RAG evaluator、reward model 和 verifier loop，并报告准确率、显存、tokens/s、延迟和失败样例。

第二，服务系统应继续跟进 MoE 和 disaggregated inference。下一代推理不只是 batch scheduler，而是涉及 prefill/decode 分离、专家路由、跨集群通信和异构资源 [23]。

第三，推理章节要持续跟踪 RLVR、GRPO、预算控制和 agentic tool use。RL for LLMs 的综述已经把 PPO、DPO、GRPO、RLHF、RLAIF 和 RLVR 放进同一设计空间 [77]。

第四，多模态与生成章节要持续跟踪视频、音频、实时语音、长上下文文档理解、统一理解-生成、扩散语言模型、AR-Diffusion 混合架构、任意到任意接口和 VLA 行动模型。Qwen3-VL 和 Qwen3-Omni 表明，多模态正从“图像加文本”走向统一交互系统 [67, 90]; Janus-Pro、MMaDA、Dream 7B、MammothModa2 和 AR-Omni 则说明，理解与生成、文本与视觉、自回归与扩散之间的边界正在重新组织 [13, 92, 95, 76, 14]; RT-2 和 OpenVLA 进一步说明，行动也可以成为统一接口的一部分 [10, 39]。

17.4 读者路线图

建议路线是：先实现最小 tokenizer 和 GPT；再加入资源核算和小规模训练；然后做数据清洗、去重和污染检查；接着做 SFT、LoRA、RAG、上下文工程和 typed memory；再做 DPO、KTO/ORPO/SimPO 对比、GRPO 或 verifier-guided RL；之后实现一个带 KV cache、流式输出、延迟统计和安全过滤的服务；最后增加一个多模态、生成式或 VLA 组件，并分别评估感知、推理、生成质量、行动安全、来源和治理。每一步都问同一个问题：目标函数是什么，数据来自哪里，系统成本是多少，评测是否可信。

17.5 深入展开：为什么结构保持 17 章

本章不是新论文列表，而是研究实践路线图。它回答一个问题：在 CS336 式从零实现课程和 2025-2026 年前沿论文之后，读者应该怎样把论文主张转化为可实现、可测量、可检查的系统？结论是，目录不应继续拆碎，而应保持生命周期结构：数据、token、架构、优化、系统、适配、检索、偏好、推理、多模态生成、评测和治理。

原因很直接。现代基础模型的创新往往发生在章节边界：MoE 改变架构，也改变训练并行和推理调度；长上下文改变模型，也改变 RAG 和评测；推理 RL 改变后训练，也改变测试时预算；统一多模态改变表示，也改变服务延迟、隐私和来源治理；VLA 改变输出模态，也改变安全责任。把这些主题拆成互不相干的小章，会让读者失去系统联系。

本书把进一步学习路线写成项目序列。先实现 tokenizer 和小 GPT，确认 shape、mask 和 loss；再加入训练循环、资源核算和 scaling；再做数据去重、污染检查和领域切片；再做 SFT、LoRA、RAG、typed memory；再做 DPO 和 verifiable RL；再做服务系统和 KV cache；最后加入多模态、生成式或 VLA 组件。每一步都要求读者报告正确性、成本和失败样例。

这一章的最终判断是：前沿会变，但不变量不会变。未来如果 attention 被替代，仍需要数据、优化、服务、适配、评测和治理；如果 GRPO 被替代，仍需要奖励设计、预算核算和鲁棒性测试；如果 Omni/VLA 系统成为主流，仍需要时间 grounding、行动安全、隐私、来源和流水线评测。书的定位越高，越不能只追模型名，而要保留这些不变量。

17.6 章节细节

17.6.1 为什么需要前沿章

大模型领域变化太快，单纯列举模型名称很快过时。前沿章的任务是给读者一套判断框架：新工作改变了目标、数据、结构、系统、推理还是评测。只有这样，读者才能持续更新知识。

17.6.2 覆盖了什么，还需要关注什么

本书覆盖文本 LLM、多模态、生成模型、对齐、RAG、agent、推理和治理，但仍有持续变化的方向。包括统一生成理解、具身智能、长程记忆、低成本推理、开放评测和制度治理。读者应把这些看作研究问题，而不是附录话题。

17.6.3 从零实践作为研究方法

从零实现 tokenizer、Transformer、训练循环、推理服务和 RAG，可以让读者理解抽象背后的约束。实践不是为了取代成熟框架，而是为了发现框架隐藏的假设。高水平研究者需要能读论文，也能看懂系统瓶颈。

17.6.4 前沿架构问题

未来架构可能在注意力、状态空间、MoE、扩散语言模型、长上下文和多模态连接器之间继续演化。判断架构贡献时，应固定数据、预算和后训练，或者明确比较的是完整系统。架构创新必须经受服务和评测检验。

17.6.5 推理与自适应测试时计算

推理能力越来越依赖测试时预算控制。研究问题包括何时多采样、何时搜索、何时调用工具、何时停止和如何校准置信度。产品系统还必须把这些选择转化为成本和延迟策略。

17.6.6 数据、评测与治理作为一条闭环

数据决定训练，评测发现问题，治理决定能否发布，监控再产生新的数据和修复需求。这是一条闭环，而不是独立章节。成熟团队会把数据 manifest、评测报告、红队结果和事故复盘连起来。

17.6.7 读者后续路线图

读者可以按四条路线深入：从零实现与系统优化、后训练与对齐、RAG/agent 应用、多模态与生成模型。每条路线都应保持同一标准：说明目标、约束、指标、数据和失败模式。能这样提问，比追逐模型名称更持久。

17.7 关键术语、实现要点与练习

关键术语。 From-scratch implementation 用最小实现暴露假设；resource accounting 报告显存、FLOPs、带宽和延迟；adaptive test-time compute 按输入难度分配推理预算；generative foundation model 生成文本、图像、音频、视频、代码或动作。

实现要点。 读者应把每个模型 claim 还原为 objective、data、architecture、training recipe、inference policy 和 evaluation protocol；新论文应被视为待测试证据，而不是直接复制的 recipe；目录结构要服务系统生命周期，而不是追逐热点。

练习。

- 1. 为本书任一章设计 CS336 式实现作业。
- 2. 比较 dense decoder 和 MoE 的总参数、激活参数、路由和服务成本。
- 3. 把第 15 章改写成 一个课程模块，安排 autoregressive、diffusion、Omni 和 VLA 实验。
- 4. 选择一篇 2025 年之后论文，说明其结论在换 tokenizer、数据混合、预算或评测 split 后可能如何改变。

17.8 前沿证据矩阵

前沿论文进入本书时，不按模型名称堆叠，而按它改变的 系统维度归档。一个新方法如果不能说明目标、数据、训练、推理和评测证据，就还只是候选线索。

前沿方向	需要追踪的变量	进入教材主线的条件
稀疏 MoE	总参数、激活参数、router、专家并行、服务延迟	质量/成本优于 dense 基线，并报告路由稳定性
长上下文与记忆	位置机制、KV 成本、检索、压缩、冲突证据	不只支持长输入，还能定位、聚合和拒绝错误证据
推理 RL 与测试时计算	奖励、采样数、验证器、token 预算、pass@k/pass@1	提升准确率的同时报告延迟、成本和轨迹风险
统一多模态生成	表示、目标函数、采样步数、来源治理	同时评测理解、生成、时间一致性和安全边界
VLA 与行动模型	观察、动作空间、控制频率、联锁、重置成本	真实或仿真任务成功率与危险动作率同时达标

可以把一篇新论文的采用价值写成带惩罚项的证据分数：

$$S_{\text{frontier}} = \Delta Q - \lambda_C \Delta C - \lambda_R \Delta R - \lambda_U U + \lambda_E E,$$

其中 ΔQ 是可复现实验中的质量增益， ΔC 是训练或服务成本增加， ΔR 是安全、隐私、版权或治理风险， U 是不确定性， E 是独立证据强度。这个公式的用途不是机械打分，而是防止只因模型名新、论文热或榜单高就改变教材结构。

附录 A 附录：记号、实验记录与术语

A.1 常用记号

B 表示 batch size, T 表示序列长度, d 表示隐藏维度, V 表示词表大小, θ 表示模型参数, π_θ 表示策略模型, p_θ 表示语言模型分布。

A.2 实验记录清单

每次实验应记录：代码版本、数据版本、tokenizer、模型配置、随机种子、优化器、学习率、batch、精度、硬件、训练时间、checkpoint、验证集、指标、失败样例和结论。

A.3 术语表

基础模型 经过预训练但尚未进行指令微调或偏好优化的模型。

后训练 在预训练之后进行的 SFT、偏好优化、安全调优和领域适配。

KV Cache 推理时缓存历史 token 的 key/value 张量以降低增量生成成本。

MoE Mixture-of-Experts, 每个 token 只激活部分专家的稀疏模型结构。

RAG Retrieval-Augmented Generation, 推理时检索外部证据并生成答案的系统。

上下文工程 对送入模型的系统指令、用户请求、检索证据、工具输出、记忆和状态进行设计、压缩、路由、权限控制和检查的工程方法。

统一理解-生成模型 同时解释输入媒体并生成或编辑媒体的多模态模型或系统。

任意到任意模型 接受多种输入模态并能输出多种模态的系统，例如文本、图像、音频、语音或视频。

扩散模型 学习反转加噪过程、通过迭代去噪生成数据的生成模型，可工作在像素、token 或 latent 空间。

Rectified Flow 学习从噪声到数据的连续流的生成建模形式，常用于高分辨率视觉生成。

VLA 模型 Vision-Language-Action 模型，根据视觉观察和语言指令在物理或仿真环境中产生动作。

世界模型 用于预测或模拟未来状态、观察或行动后果的模型，可辅助规划、评测或控制。

RLVR Reinforcement Learning with Verifiable Rewards，用可验证答案、代码测试或规则奖励训练模型。

Unlearning 训练后删除或压制特定知识或行为，同时尽量保留无关能力的技术方向。

水印 在生成内容中嵌入或检测统计信号、元数据或来源标记的方法。

内容凭证 描述媒体来源、编辑历史或生成过程的签名来源元数据。

评测污染 训练或开发流程接触到测试题、答案、解析或变体，导致分数高估。

参考文献

- [1] Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., Sanghai, S.: Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245 (2023). URL <https://arxiv.org/abs/2305.13245>
- [2] Alayrac, J.B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., Lenc, K., Mensch, A., Millican, K., Reynolds, M., Ring, R., Rutherford, E., Cabi, S., Han, T., Gong, Z., Samangooei, S., Monteiro, M., Menick, J., Borgeaud, S., Brock, A., Nematzadeh, A., Sharifzadeh, S., Binkowski, M., Barreira, R., Vinyals, O., Zisserman, A., Simonyan, K.: Flamingo: a visual language model for few-shot learning. arXiv preprint arXiv:2204.14198 (2022). URL <https://arxiv.org/abs/2204.14198>
- [3] Alomrani, M.A., Zhang, Y., Li, D., Sun, Q., Pal, S., Zhang, Z., Hu, Y., Ajwani, R.D., Valkanas, A., Karimi, R., et al.: Reasoning on a budget: A survey of adaptive and controllable test-time compute in llms. arXiv preprint arXiv:2507.02076 (2025). URL <https://arxiv.org/abs/2507.02076>
- [4] Anthropic: Circuit tracing: Revealing computational graphs in language models. Transformer Circuits Thread (2025). URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>
- [5] Anthropic: Responsible scaling policy version 3.0. Tech. rep., Anthropic (2026). URL <https://www.anthropic.com/news/responsible-scaling-policy-v3>
- [6] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al.: Constitutional ai: Harmlessness from ai feedback. arXiv preprint arXiv:2212.08073 (2022). URL <https://arxiv.org/abs/2212.08073>
- [7] Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., Chen, A., Goldie, A., Mirhoseini, A., McKinnon, C., et al.: Training a helpful and harmless assistant

- with reinforcement learning from human feedback. arXiv preprint arXiv:2204.05862 (2022). URL <https://arxiv.org/abs/2204.05862>
- [8] Bai, Y., Lv, X., Zhang, J., Lyu, H., Tang, J., Huang, Z., Du, Z., Liu, X., Zeng, A., Hou, L., et al.: Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv preprint arXiv:2412.15204 (2024). URL <https://arxiv.org/abs/2412.15204>
- [9] Behrouz, A., Zhong, P., Mirrokni, V.: Titans: Learning to memorize at test time. arXiv preprint arXiv:2501.00663 (2025). URL <https://arxiv.org/abs/2501.00663>
- [10] Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al.: Rt-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818 (2023). URL <https://arxiv.org/abs/2307.15818>
- [11] Chandra, B., Dunietz, J., Roberts, K., Lee, Y., Fontana, P., Awad, G.: Reducing risks posed by synthetic content: An overview of technical approaches to digital content transparency. Tech. Rep. NIST AI 100-4, National Institute of Standards and Technology (2024). URL <https://doi.org/10.6028/NIST.AI.100-4>
- [12] Chen, J., Meng, S., Chen, Y., Zhao, M., Gui, W., Guo, X.: Toc-bench: A temporal object consistency benchmark for video large language models. arXiv preprint arXiv:2605.09904 (2026). URL <https://arxiv.org/abs/2605.09904>
- [13] Chen, X., Wu, Z., Liu, X., Pan, Z., Liu, W., Xie, Z., Yu, X., Ruan, C.: Janus-pro: Unified multimodal understanding and generation with data and model scaling. arXiv preprint arXiv:2501.17811 (2025). URL <https://arxiv.org/abs/2501.17811>
- [14] Cheng, D., Yuan, R., Li, Y., You, R., Wang, W., Nie, L., Zhang, L., Li, W.: Ar-omni: A unified autoregressive model for any-to-any generation. arXiv preprint arXiv:2601.17761 (2026). URL <https://arxiv.org/abs/2601.17761>
- [15] Coalition for Content Provenance and Authenticity: C2pa specifications. Technical specification (2026). URL <https://spec.c2pa.org/specifications/specifications/2.0/index.html>
- [16] Dao, T., Gu, A.: Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. arXiv preprint arXiv:2405.21060 (2024). URL <https://arxiv.org/abs/2405.21060>

- [17] DeepSeek-AI, Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Xu, R., Zhu, Q., Zhang, R., Ma, S., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025). URL <https://arxiv.org/abs/2501.12948>
- [18] DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Ruan, C., et al.: Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437 (2024). URL <https://arxiv.org/abs/2412.19437>
- [19] Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L.: Qlora: Efficient finetuning of quantized llms. arXiv preprint arXiv:2305.14314 (2023). URL <https://arxiv.org/abs/2305.14314>
- [20] Esser, P., Kulal, S., Blattmann, A., Entezari, R., Müller, J., Saini, H., Levi, Y., Lorenz, D., Sauer, A., Boesel, F., et al.: Scaling rectified flow transformers for high-resolution image synthesis. arXiv preprint arXiv:2403.03206 (2024). URL <https://arxiv.org/abs/2403.03206>
- [21] Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., Kiela, D.: Kto: Model alignment as prospect theoretic optimization. arXiv preprint arXiv:2402.01306 (2024). URL <https://arxiv.org/abs/2402.01306>
- [22] European Union: Regulation (eu) 2024/1689 of the european parliament and of the council (artificial intelligence act) (2024). URL <https://eur-lex.europa.eu/legal-content/en/TXT/?uri=CELEX:32024R1689>
- [23] Feng, Y., Tan, X., Sew, K.H., Jiang, Y., Zhu, Y., Xu, H.: Frontier: Simulating the next generation of llm inference systems. arXiv preprint arXiv:2508.03148 (2025). URL <https://arxiv.org/abs/2508.03148>
- [24] Gan, A., Yu, H., Zhang, K., Liu, Q., Yan, W., Huang, Z., Tong, S., Hu, G.: Retrieval augmented generation evaluation in the era of large language models: A comprehensive survey. arXiv preprint arXiv:2504.14891 (2025). URL <https://arxiv.org/abs/2504.14891>
- [25] Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al.: The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024). URL <https://arxiv.org/abs/2407.21783>

- [26] Gu, A., Dao, T.: Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752 (2023). URL <https://arxiv.org/abs/2312.00752>
- [27] Guan, T., Liu, F., Wu, X., Xian, R., Li, Z., Liu, X., Wang, X., Chen, L., Huang, F., Yacoub, Y., Manocha, D., Zhou, T.: Hallusionbench: An advanced diagnostic suite for entangled language hallucination and visual illusion in large vision-language models. arXiv preprint arXiv:2310.14566 (2023). URL <https://arxiv.org/abs/2310.14566>
- [28] Guo, X., Huo, J., Shi, Z., Song, Z., Zhang, J., Zhao, J.: T2vphysbench: A first-principles benchmark for physical consistency in text-to-video generation. arXiv preprint arXiv:2505.00337 (2025). URL <https://arxiv.org/abs/2505.00337>
- [29] Gururangan, S., Marasovic, A., Swayamdipta, S., Lo, K., Beltagy, I., Downey, D., Smith, N.A.: Don’t stop pretraining: Adapt language models to domains and tasks. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (2020). URL <https://arxiv.org/abs/2004.10964>
- [30] Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., Steinhardt, J.: Measuring massive multitask language understanding. arXiv preprint arXiv:2009.03300 (2020). URL <https://arxiv.org/abs/2009.03300>
- [31] Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L.A., Welbl, J., Clark, A., et al.: Training compute-optimal large language models. arXiv preprint arXiv:2203.15556 (2022). URL <https://arxiv.org/abs/2203.15556>
- [32] Hong, J., Lee, N., Thorne, J.: Orpo: Monolithic preference optimization without reference model. arXiv preprint arXiv:2403.07691 (2024). URL <https://arxiv.org/abs/2403.07691>
- [33] Hsieh, C.P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Ginsburg, B.: Ruler: What’s the real context size of your long-context language models? arXiv preprint arXiv:2404.06654 (2024). URL <https://arxiv.org/abs/2404.06654>
- [34] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W.: Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685 (2021). URL <https://arxiv.org/abs/2106.09685>

- [35] Huang, Z., Zhang, F., Xu, X., He, Y., Yu, J., Dong, Z., Ma, Q., Chanpaisit, N., Si, C., Jiang, Y., et al.: Vbench++: Comprehensive and versatile benchmark suite for video generative models. arXiv preprint arXiv:2411.13503 (2024). URL <https://arxiv.org/abs/2411.13503>
- [36] Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: Swe-bench: Can language models resolve real-world github issues? arXiv preprint arXiv:2310.06770 (2023). URL <https://arxiv.org/abs/2310.06770>
- [37] Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D.: Scaling laws for neural language models. arXiv preprint arXiv:2001.08361 (2020). URL <https://arxiv.org/abs/2001.08361>
- [38] Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. arXiv preprint arXiv:2004.04906 (2020). URL <https://arxiv.org/abs/2004.04906>
- [39] Kim, M.J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al.: Openvla: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246 (2024). URL <https://arxiv.org/abs/2406.09246>
- [40] Kimi Team, Bai, Y., Bao, Y., Charles, Y., Chen, C., Chen, G., Chen, H., Chen, J., Chen, N., Chen, R., et al.: Kimi k2: Open agentic intelligence. arXiv preprint arXiv:2507.20534 (2025). URL <https://arxiv.org/abs/2507.20534>
- [41] Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I., Goldstein, T.: A watermark for large language models. In: International Conference on Machine Learning (2023). URL <https://arxiv.org/abs/2301.10226>
- [42] Kudo, T., Richardson, J.: Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. arXiv preprint arXiv:1808.06226 (2018). URL <https://arxiv.org/abs/1808.06226>
- [43] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I.: Efficient memory management for large language model serving with pagedattention. arXiv preprint arXiv:2309.06180 (2023). URL <https://arxiv.org/abs/2309.06180>

- [44] Lee, H., Phatale, S.M.X., Lu, K., Mesnard, T., Bishop, C., Carbune, V., Rastogi, A.: Rlaif: Scaling reinforcement learning from human feedback with ai feedback. arXiv preprint arXiv:2309.00267 (2023). URL <https://arxiv.org/abs/2309.00267>
- [45] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.t., Rocktaschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. arXiv preprint arXiv:2005.11401 (2020). URL <https://arxiv.org/abs/2005.11401>
- [46] Li, J., Li, D., Savarese, S., Hoi, S.: Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. arXiv preprint arXiv:2301.12597 (2023). URL <https://arxiv.org/abs/2301.12597>
- [47] Li, Y., Du, Y., Zhou, K., Wang, J., Zhao, W.X., Wen, J.R.: Evaluating object hallucination in large vision-language models. arXiv preprint arXiv:2305.10355 (2023). URL <https://arxiv.org/abs/2305.10355>
- [48] Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., Zhang, Y., Narayanan, D., Wu, Y., Kumar, A., Newman, B., Yuan, B., Yan, B., Zhang, C., Cosgrove, C., Manning, C.D., Re, C., Acosta-Navas, D., Hudson, D.A., Zelikman, E., Durmus, E., Ladhak, F., Rong, F., Ren, H., Yao, H., Wang, J., Santhanam, K., Orr, L., Zheng, L., Yuksekgonul, M., Suzgun, M., Kim, N., Guha, N., Chatterji, N., Khattab, O., Henderson, P., Huang, Q., Chi, R., Xie, S.M., Santurkar, S., Ganguli, S., Hashimoto, T., Icard, T., Zhang, T., Chaudhary, V., Wang, W., Li, X., Mai, Y., Zhang, Y., Koreeda, Y.: Holistic evaluation of language models. arXiv preprint arXiv:2211.09110 (2022). URL <https://arxiv.org/abs/2211.09110>
- [49] Liu, H., Li, C., Wu, Q., Lee, Y.J.: Visual instruction tuning. arXiv preprint arXiv:2304.08485 (2023). URL <https://arxiv.org/abs/2304.08485>
- [50] Liu, X., Zhu, Y., Gu, J., Lan, Y., Yang, C., Qiao, Y.: Mm-safetybench: A benchmark for safety evaluation of multimodal large language models. arXiv preprint arXiv:2311.17600 (2023). URL <https://arxiv.org/abs/2311.17600>
- [51] Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., Zhu, C.: G-eval: Nlg evaluation using gpt-4 with better human alignment. arXiv preprint arXiv:2303.16634 (2023). URL <https://arxiv.org/abs/2303.16634>

- [52] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. International Conference on Learning Representations (2019). URL <https://arxiv.org/abs/1711.05101>
- [53] Lu, P., Bansal, H., Xia, T., Liu, J., Li, C., Hajishirzi, H., Cheng, H., Chang, K.W., Galley, M., Gao, J.: Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. arXiv preprint arXiv:2310.02255 (2023). URL <https://arxiv.org/abs/2310.02255>
- [54] Mei, L., Yao, J., Ge, Y., Wang, Y., Bi, B., Cai, Y., Liu, J., Li, M., Li, Z.Z., Zhang, D., et al.: A survey of context engineering for large language models. arXiv preprint arXiv:2507.13334 (2025). URL <https://arxiv.org/abs/2507.13334>
- [55] Meng, Y., Xia, M., Chen, D.: Simpo: Simple preference optimization with a reference-free reward. arXiv preprint arXiv:2405.14734 (2024). URL <https://arxiv.org/abs/2405.14734>
- [56] Mukherjee, S., Mitra, A., Jawahar, G., Agarwal, S., Palangi, H., Awadallah, A.: Orca: Progressive learning from complex explanation traces of gpt-4. arXiv preprint arXiv:2306.02707 (2023). URL <https://arxiv.org/abs/2306.02707>
- [57] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Saunders, W., et al.: Webgpt: Browser-assisted question-answering with human feedback. arXiv preprint arXiv:2112.09332 (2021). URL <https://arxiv.org/abs/2112.09332>
- [58] National Institute of Standards and Technology: Artificial intelligence risk management framework (ai rmf 1.0). Tech. rep., National Institute of Standards and Technology (2023). URL <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf>
- [59] National Institute of Standards and Technology: Artificial intelligence risk management framework: Generative artificial intelligence profile. Tech. rep., National Institute of Standards and Technology (2024). URL <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>
- [60] OpenAI: Video generation models as world simulators. Technical report (2024). URL <https://openai.com/research/video-generation-models-as-world-simulators>

- [61] OpenAI: Browsecomp: A simple yet challenging benchmark for browsing agents. arXiv preprint arXiv:2504.12516 (2025). URL <https://arxiv.org/abs/2504.12516>
- [62] OpenAI: Our updated preparedness framework. Tech. rep., OpenAI (2025). URL <https://openai.com/index/updating-our-preparedness-framework/>
- [63] Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C.L., Mishkin, P., Zhang, C., Agarwal, S., Slama, K., Ray, A., et al.: Training language models to follow instructions with human feedback. arXiv preprint arXiv:2203.02155 (2022). URL <https://arxiv.org/abs/2203.02155>
- [64] Peebles, W., Xie, S.: Scalable diffusion models with transformers. Proceedings of the IEEE/CVF International Conference on Computer Vision (2023). URL <https://arxiv.org/abs/2212.09748>
- [65] Phan, L., Gatti, A., Han, Z., Li, N., Hu, J., Zhang, H., Zhang, C.B.C., Shaa-ban, M., Ling, J., Shi, S., Choi, M., et al.: Humanity’s last exam. arXiv preprint arXiv:2501.14249 (2025). URL <https://arxiv.org/abs/2501.14249>
- [66] Qiu, R., Tan, J., Pu, J., Wang, H., Gao, X.S., Sun, F.: A survey on unlearning in large language models. arXiv preprint arXiv:2510.25117 (2025). URL <https://arxiv.org/abs/2510.25117>
- [67] Qwen Team: Qwen3-vl technical report. arXiv preprint arXiv:2511.21631 (2025). URL <https://arxiv.org/abs/2511.21631>
- [68] Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: Learning transferable visual models from natural language supervision. Proceedings of the 38th International Conference on Machine Learning (2021). URL <https://arxiv.org/abs/2103.00020>
- [69] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., Finn, C.: Direct preference optimization: Your language model is secretly a reward model. arXiv preprint arXiv:2305.18290 (2023). URL <https://arxiv.org/abs/2305.18290>
- [70] Rajbhandari, S., Rasley, J., Ruwase, O., He, Y.: Zero: Memory optimizations toward training trillion parameter models. Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (2020). URL <https://arxiv.org/abs/1910.02054>

- [71] Rein, D., Hou, B.L., Stickland, A.C., Petty, J., Pang, R.Y., Dirani, J., Michael, J., Bowman, S.R.: Gpqa: A graduate-level google-proof q&a benchmark. arXiv preprint arXiv:2311.12022 (2023). URL <https://arxiv.org/abs/2311.12022>
- [72] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems (2023). URL <https://arxiv.org/abs/2302.04761>
- [73] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347 (2017). URL <https://arxiv.org/abs/1707.06347>
- [74] Sennrich, R., Haddow, B., Birch, A.: Neural machine translation of rare words with subword units. arXiv preprint arXiv:1508.07909 (2016). URL <https://arxiv.org/abs/1508.07909>
- [75] Shazeer, N.: Glu variants improve transformer. arXiv preprint arXiv:2002.05202 (2020). URL <https://arxiv.org/abs/2002.05202>
- [76] Shen, T., Wan, X., Chen, T., Zhang, R., Pan, J., Lu, D., Lei, F., Lu, Z., Yang, Y., Cheng, C., et al.: Mammothmoda2: A unified ar-diffusion framework for multimodal understanding and generation. arXiv preprint arXiv:2511.18262 (2025). URL <https://arxiv.org/abs/2511.18262>
- [77] Srivastava, S.S., Aggarwal, V.: A technical survey of reinforcement learning techniques for large language models. arXiv preprint arXiv:2507.04136 (2025). URL <https://arxiv.org/abs/2507.04136>
- [78] Stanford University: CS336: Language modeling from scratch. Course website (2026). URL <https://cs336.stanford.edu/>. Accessed 2026-05-20
- [79] Stiennon, N., Ouyang, L., Wu, J., Ziegler, D.M., Lowe, R., Voss, C., Radford, A., Amodei, D., Christiano, P.F.: Learning to summarize with human feedback. Advances in Neural Information Processing Systems (2020). URL <https://arxiv.org/abs/2009.01325>
- [80] Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., Wei, F.: Retentive network: A successor to transformer for large language models. arXiv preprint arXiv:2307.08621 (2023). URL <https://arxiv.org/abs/2307.08621>

- [81] Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., Pearce, A., Citro, C., Ameisen, E., Jones, A., et al.: Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. Transformer Circuits Thread (2024). URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/>
- [82] Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.A., Lacroix, T., Roziere, B., Goyal, N., Hambro, E., Azhar, F., et al.: Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023). URL <https://arxiv.org/abs/2302.13971>
- [83] Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., et al.: Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023). URL <https://arxiv.org/abs/2307.09288>
- [84] Ud Din, M., Akram, W., Saoud, L.S., Rosell, J., Hussain, I.: Vision language action models in robotic manipulation: A systematic review. arXiv preprint arXiv:2507.10672 (2025). URL <https://arxiv.org/abs/2507.10672>
- [85] Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E.H., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. arXiv preprint arXiv:2203.11171 (2022). URL <https://arxiv.org/abs/2203.11171>
- [86] Wang, Y., Kordi, Y., Mishra, S., Liu, A., Smith, N.A., Khashabi, D., Hajishirzi, H.: Self-instruct: Aligning language models with self-generated instructions. arXiv preprint arXiv:2212.10560 (2022). URL <https://arxiv.org/abs/2212.10560>
- [87] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. arXiv preprint arXiv:2201.11903 (2022). URL <https://arxiv.org/abs/2201.11903>
- [88] Xu, C., Guan, S., Greene, D., Kechadi, M.T.: Benchmark data contamination of large language models: A survey. arXiv preprint arXiv:2406.04244 (2024). URL <https://arxiv.org/abs/2406.04244>
- [89] Xu, C., Sun, Q., Zheng, K., Geng, X., Zhao, P., Feng, J., Tao, C., Jiang, D.: Wizardlm: Empowering large language models to follow complex instructions. arXiv preprint arXiv:2304.12244 (2023). URL <https://arxiv.org/abs/2304.12244>

- [90] Xu, J., Guo, Z., Hu, H., Chu, Y., Wang, X., He, J., Wang, Y., Shi, X., He, T., Zhu, X., et al.: Qwen3-omni technical report. arXiv preprint arXiv:2509.17765 (2025). URL <https://arxiv.org/abs/2509.17765>
- [91] Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al.: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025). URL <https://arxiv.org/abs/2505.09388>
- [92] Yang, L., Tian, Y., Li, B., Zhang, X., Shen, K., Tong, Y., Wang, M.: Mmada: Multimodal large diffusion language models. arXiv preprint arXiv:2505.15809 (2025). URL <https://arxiv.org/abs/2505.15809>
- [93] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T.L., Cao, Y., Narasimhan, K.: Tree of thoughts: Deliberate problem solving with large language models. arXiv preprint arXiv:2305.10601 (2023). URL <https://arxiv.org/abs/2305.10601>
- [94] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: React: Synergizing reasoning and acting in language models. arXiv preprint arXiv:2210.03629 (2023). URL <https://arxiv.org/abs/2210.03629>
- [95] Ye, J., Xie, Z., Zheng, L., Gao, J., Wu, Z., Jiang, X., Li, Z., Kong, L.: Dream 7b: Diffusion large language models. arXiv preprint arXiv:2508.15487 (2025). URL <https://arxiv.org/abs/2508.15487>
- [96] Yue, X., Ni, Y., Zhang, K., Zheng, T., Liu, R., Zhang, G., Stevens, S., Jiang, D., Ren, W., Sun, Y., Wei, C., Yu, B., Yuan, R., Sun, R., Yin, M., Zheng, B., Yang, Z., Liu, Y., Huang, W., Sun, H., Su, Y., Chen, W.: Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. arXiv preprint arXiv:2311.16502 (2023). URL <https://arxiv.org/abs/2311.16502>
- [97] Zelikman, E., Wu, Y., Mu, J., Goodman, N.: Star: Bootstrapping reasoning with reasoning. Advances in Neural Information Processing Systems (2022). URL <https://arxiv.org/abs/2203.14465>
- [98] Zhang, Q., Lyu, F., Sun, Z., Wang, L., Zhang, W., Guo, Z., Wang, Y., King, I., Liu, X., Ma, C.: What, how, where, and how well? a survey on test-time scaling in large language models. arXiv preprint arXiv:2503.24235 (2025). URL <https://arxiv.org/abs/2503.24235>

- [99] Zhang, X., Chen, Y., Hu, S., Xu, Z., Chen, J., Hao, M.K., Han, X., Thai, Z.L., Wang, S., Liu, Z., Sun, M.: ∞ bench: Extending long context evaluation beyond 100k tokens. arXiv preprint arXiv:2402.13718 (2024). URL <https://arxiv.org/abs/2402.13718>
- [100] Zhao, S., Zhang, X., Guo, J., Hu, J., Duan, L., Fu, M., Chng, Y.X., Wang, G.H., Chen, Q.G., Xu, Z., et al.: Unified multimodal understanding and generation models: Advances, challenges, and opportunities. arXiv preprint arXiv:2505.02567 (2025). URL <https://arxiv.org/abs/2505.02567>
- [101] Zhao, Y., Gu, A., Varma, R., Luo, L., Huang, C.C., Xu, M., Wright, L., Shojanazeri, H., Ott, M., Shleifer, S., Desmaison, A., Balioglu, C., Damania, P., Nguyen, B., Chauhan, G., Hao, Y., Mathews, A., Li, S.: Pytorch fsdp: Experiences on scaling fully sharded data parallel. Proceedings of the VLDB Endowment (2023). URL <https://arxiv.org/abs/2304.11277>
- [102] Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging llm-as-a-judge with mt-bench and chatbot arena. arXiv preprint arXiv:2306.05685 (2023). URL <https://arxiv.org/abs/2306.05685>
- [103] Ziegler, D.M., Stiennon, N., Wu, J., Brown, T.B., Radford, A., Amodei, D., Christiano, P., Irving, G.: Fine-tuning language models from human preferences. arXiv preprint arXiv:1909.08593 (2019). URL <https://arxiv.org/abs/1909.08593>